

Big Data Mid Sem Sandali

1. Numericals:
 - a. K Means
 - b. DT
 - c. Naive Bayes
 - d. Classification

1. Characteristics of Big Data

Big Data is usually explained by the **5 V's**:

1. **Volume** – Refers to the huge amount of data generated every second from social media, IoT devices, transactions, sensors, etc.
 2. **Velocity** – Data is generated and processed at very high speed (e.g., live streams, stock markets, online transactions).
 3. **Variety** – Data comes in multiple formats: structured (databases), semi-structured (XML, JSON), and unstructured (videos, images, text).
 4. **Veracity** – Refers to the reliability and accuracy of data; big data may have inconsistencies, noise, or uncertainty.
 5. **Value** – The most important aspect; extracting meaningful insights from raw data to create business value.
-

Transformation from Traditional Approach to Big Data Approach

1. Data Storage

- *Traditional*: Relied on relational databases (RDBMS) and limited storage capacity.
- *Big Data*: Uses distributed storage systems (Hadoop, Cloud) that handle petabytes/exabytes.

2. Data Processing

- *Traditional:* Batch processing, slower, suitable only for structured data.
- *Big Data:* Real-time processing with tools like Spark and Flink, works with structured + unstructured data.

3. Decision Making

- *Traditional:* Decisions based on historical reports and limited datasets.
- *Big Data:* Data-driven, predictive analytics, real-time dashboards enable faster and smarter decisions.

4. Cost and Scalability

- *Traditional:* Expensive and less scalable due to fixed infrastructure.
- *Big Data:* Cloud-based and distributed systems reduce cost, easily scalable with growing data.

5. Business Impact

- *Traditional:* Focused on past trends, reactive strategies.
- *Big Data:* Enables personalization (e.g., Amazon, Netflix), fraud detection, targeted marketing, proactive strategies.

2. Big Data Technologies and Tools for Ride-Sharing Company

A ride-sharing company generates huge amounts of data from GPS devices, customer ride requests, traffic sensors, and driver availability. To handle this, we need **Big Data technologies** for **storage, processing, and analytics**.

1. Data Storage

- **Technology/Tool:**
 - **Hadoop Distributed File System (HDFS)**
 - **Cloud Storage (AWS S3, Google BigQuery, Azure Data Lake)**

- **Justification:**
 - Stores massive volumes of structured (ride logs, transactions) and unstructured (GPS streams, traffic maps) data.
 - Scalable and fault-tolerant, ensuring continuous availability of data for analytics.
-

2. Data Processing

- **Technology/Tool:**
 - **Apache Spark** (for real-time + batch processing)
 - **Hadoop MapReduce** (for large-scale batch jobs)
 - **Apache Kafka / Spark Streaming** (for real-time event streaming)
 - **Justification:**
 - Spark enables **fast in-memory computation** for analyzing historical ride patterns (demand forecasting).
 - Kafka + Spark Streaming handle **real-time GPS and traffic feeds** to suggest optimal routes and driver assignments instantly.
 - Processing both historical and real-time data improves responsiveness and efficiency.
-

3. Data Analytics & Machine Learning

- **Technology/Tool:**
 - **Spark MLlib / Scikit-learn / TensorFlow**
 - **GIS tools (for route optimization)**
- **Justification:**

- **Demand Forecasting:** Predicts ride demand using ML models trained on historical ride data (time, location, events).
 - **Route Optimization:** Uses traffic data + graph algorithms to find shortest/fastest routes, minimizing time and fuel.
 - **Driver Assignment:** Real-time analytics ensures customers are matched with nearest available drivers, reducing wait time.
-

4. Business Impact

- Improves **customer satisfaction** through shorter wait times.
- Reduces **operational costs** by optimizing fuel consumption.
- Enables **data-driven decisions** through predictive and real-time analytics.
- Provides a **competitive advantage** compared to traditional systems that only use static, historical reporting.

3. Hadoop Distributed File System (HDFS)

HDFS is the primary storage system of the Hadoop framework. It is designed to store and manage **large-scale data** efficiently in a distributed environment.

Characteristics of HDFS:

1. **Distributed Storage** – Data is split into fixed-size blocks (default 128 MB/256 MB) and distributed across multiple machines in a cluster.
2. **Replication for Fault Tolerance** – Each block is replicated (default 3 copies) across different nodes to ensure no data loss in case of failure.
3. **Scalability** – New nodes can be easily added to increase storage capacity without disrupting existing data.
4. **High Throughput** – Optimized for reading and writing large files, making it suitable for batch processing.

5. **Write Once, Read Many** – Data in HDFS is written once and read many times, ensuring data consistency and fast access.
-

Architecture Components:

1. **NameNode (Master):**

- Maintains metadata (file names, block locations).
- Controls access to files but does not store actual data.

2. **DataNodes (Slaves):**

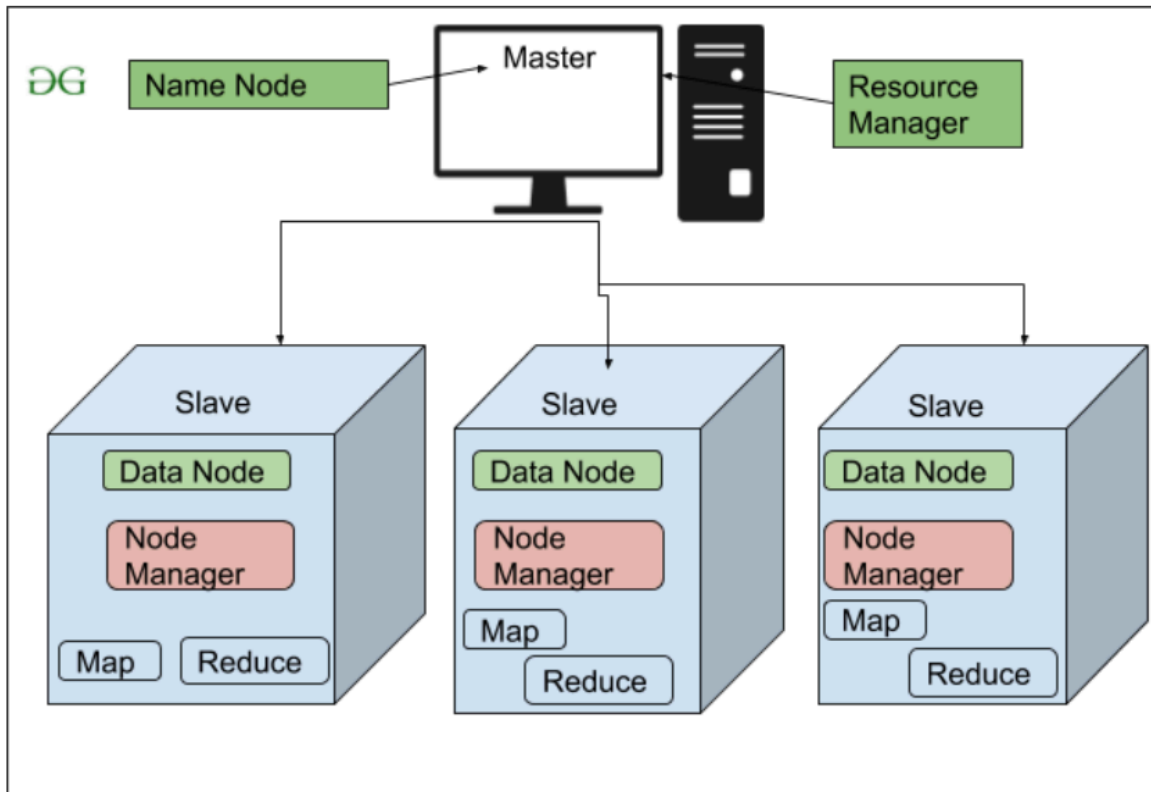
- Store actual data blocks.
- Perform read/write requests from clients.

3. **Secondary NameNode:**

- Periodically merges metadata and prevents NameNode overload.
-

Advantages of HDFS:

- Handles **huge datasets** (terabytes to petabytes).
- **Fault-tolerant** due to block replication.
- **Cost-effective** as it uses commodity hardware.
- Forms the backbone of big data ecosystems like Hadoop, Spark, Hive, etc.



4. MapReduce

MapReduce is a **programming model** and **processing engine** in the Hadoop framework used for **processing large-scale data** in parallel across distributed clusters. It divides a task into smaller sub-tasks and executes them simultaneously, ensuring speed and efficiency.

Phases of MapReduce:

1. Map Phase

- Input data is split into key-value pairs.
- The mapper processes these pairs and generates intermediate key-value pairs.
- Example: For word count, the mapper takes a line of text and outputs (word, 1).

2. Shuffle & Sort Phase

- The intermediate results from the mappers are grouped and sorted by key.
- Ensures that all values for the same key are collected together.

3. Reduce Phase

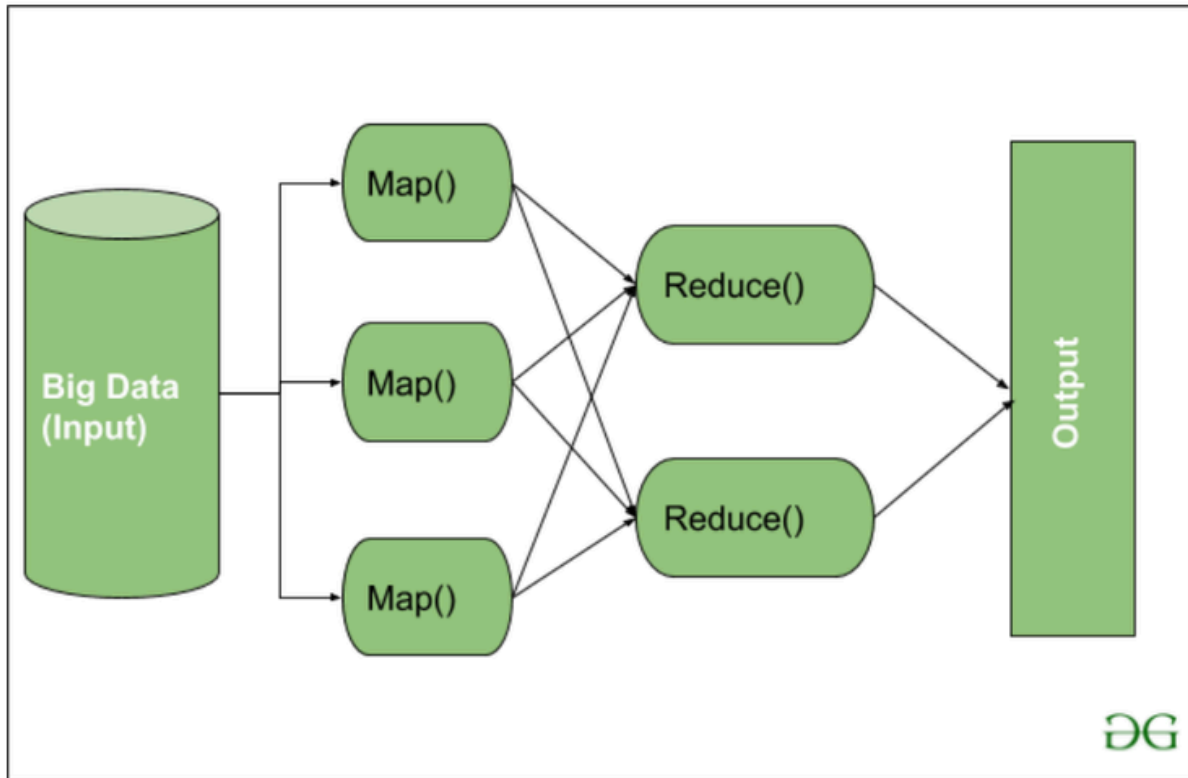
- The reducer processes grouped key–value pairs and produces the final output.
- Example: (word, [1,1,1]) → (word, 3) gives the total word frequency.

Characteristics:

- **Parallel Processing:** Tasks are distributed across multiple nodes for faster execution.
- **Fault Tolerance:** Failed tasks are automatically reassigned to other nodes.
- **Scalability:** Can process terabytes to petabytes of data by adding more nodes.
- **Simplicity:** Developers only need to write `map()` and `reduce()` functions; Hadoop handles distribution and coordination.

Advantages:

- Efficient for **batch processing** of huge datasets.
- Runs on **commodity hardware**, making it cost-effective.
- Provides **high fault tolerance** and **data locality optimization** (processing data near where it is stored).



5. YARN (Yet Another Resource Negotiator)

YARN is the **resource management layer** of the Hadoop ecosystem. It was introduced in Hadoop 2.0 to overcome the limitations of the original MapReduce framework. YARN manages **resources** (CPU, memory) and **schedules jobs** across a Hadoop cluster, allowing multiple applications to run simultaneously.

Main Components of YARN:

1. Resource Manager (RM):

- Runs on the master node.
- Allocates cluster resources to applications.
- Has two parts:

- **Scheduler** – Allocates resources based on policies (FIFO, Capacity, Fair Scheduling).
- **Application Manager** – Manages job submissions and monitoring.

2. **Node Manager (NM):**

- Runs on each worker node.
- Reports resource availability (CPU, RAM) to the Resource Manager.
- Manages the execution of tasks (containers) on the node.

3. **Application Master (AM):**

- Launched per application.
- Negotiates resources with RM and coordinates tasks with NMs.

4. **Containers:**

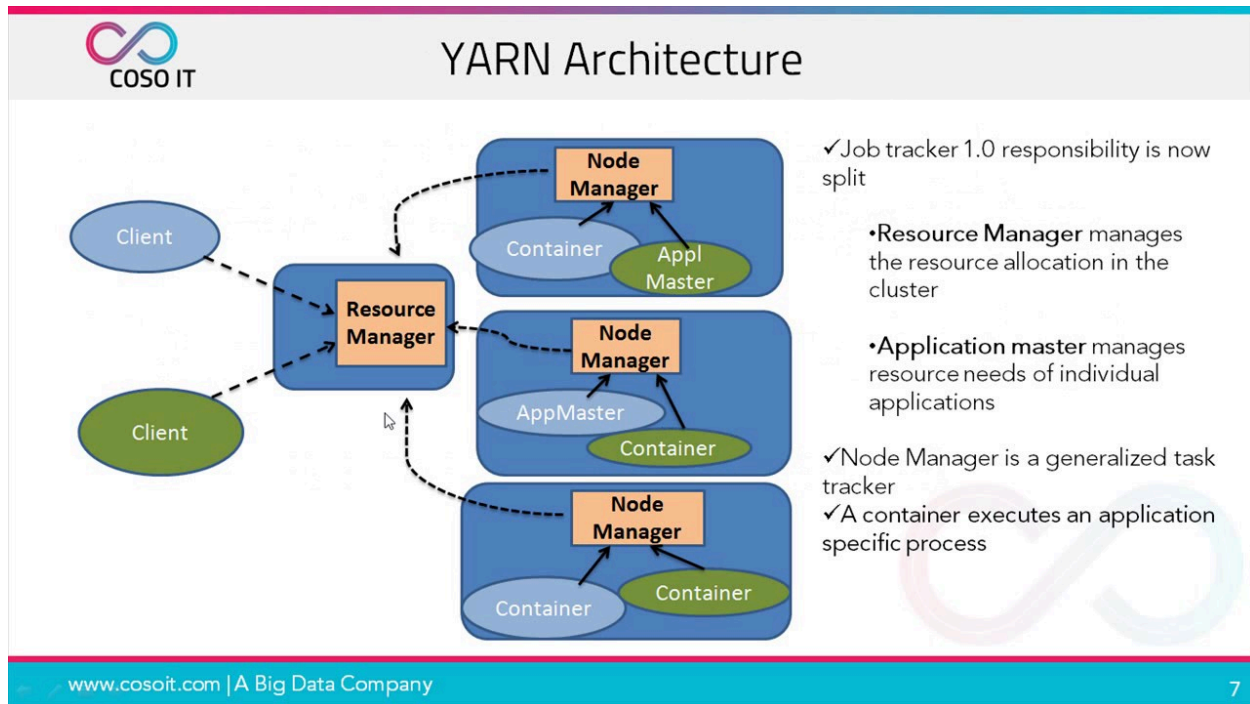
- Logical units that hold resources (CPU, memory).
- All tasks run inside containers.

Characteristics of YARN:

- **Multi-tenancy:** Supports multiple processing frameworks (MapReduce, Spark, Tez, etc.).
- **Scalability:** Efficiently handles thousands of nodes and applications.
- **Resource Utilization:** Allocates resources dynamically to maximize cluster usage.
- **Fault Tolerance:** Restarts failed containers or AMs automatically.

Advantages:

- Overcomes limitation of MapReduce-only processing.
- Enables **real-time, interactive, and batch processing** on the same cluster.
- Improves **cluster efficiency** and **resource sharing**.
- Provides **flexibility** by supporting diverse applications beyond MapReduce.



Practice Question (leave it for now):

A large e-commerce company wants to improve customer experience by analyzing massive amounts of customer clickstream data, purchase history, and product reviews. Explain which Big Data technologies and tools can be used for storage, processing, and analytics. How will these tools help in improving business decisions compared to traditional methods?

Answer:

1. Data Storage

- **Technology/Tools:**
 - **HDFS or Cloud Storage (AWS S3, Azure Data Lake, Google BigQuery)**
 - **Justification:**
 - Stores massive clickstream logs (structured/semi-structured) and unstructured product reviews.
 - Provides scalability and fault tolerance for handling growing data volumes.
-

2. Data Processing

- **Technology/Tools:**
 - **Apache Spark** (real-time + batch processing)
 - **Apache Kafka** (real-time streaming of customer activities)
 - **Justification:**
 - Spark processes purchase history and reviews for batch analytics.
 - Kafka + Spark Streaming captures real-time user clicks, searches, and cart additions to recommend products instantly.
-

3. Data Analytics & Machine Learning

- **Technology/Tools:**
 - **Spark MLlib, TensorFlow, or Scikit-learn**
 - **Natural Language Processing (NLP) tools** for sentiment analysis of product reviews
- **Justification:**

- **Customer Segmentation:** Clustering algorithms to divide customers into groups (frequent buyers, discount seekers, premium shoppers).
 - **Recommendation System:** Collaborative filtering + ML to recommend products (like Amazon, Flipkart).
 - **Sentiment Analysis:** NLP on product reviews to understand customer opinions and improve products.
-

4. Business Benefits (Compared to Traditional Systems):

- **Personalized Marketing:** Customers see relevant product suggestions, boosting sales.
- **Faster Decision-Making:** Real-time analytics vs. traditional delayed reports.
- **Improved Customer Satisfaction:** Quick response to negative reviews and feedback.
- **Scalability:** Easily handles millions of daily transactions, unlike traditional RDBMS.
- **Competitive Advantage:** Data-driven strategies attract and retain more customers.

6. Predictive Analysis for Forecasting Ride Demand in City Zones

1. Objective

To forecast the **number of ride requests** in different city zones (e.g., residential, business, airports) so that drivers can be positioned in advance, reducing wait time and improving efficiency.

2. Data Collection

- **Historical Ride Data:** Time, location (zone), trip start & end, ride counts.
- **External Data:** Weather conditions, holidays, special events, traffic data.

- **Real-Time Streams:** GPS locations, ongoing ride requests.
-

3. Data Preprocessing

- **Data Cleaning:** Handle missing/incomplete trip records.
 - **Feature Engineering:**
 - Time features (hour of day, day of week, season).
 - Zone-based features (residential, commercial, event areas).
 - External features (rain, temperature, holiday indicator).
 - **Aggregation:** Aggregate ride requests per zone per time interval (e.g., hourly).
-

4. Model Selection & Approach

1. **Time-Series Forecasting Models:**
 - **ARIMA/SARIMA:** Captures seasonality and trends in ride demand.
 - **LSTM (Recurrent Neural Network):** Handles sequential patterns for demand forecasting.
 2. **Supervised Machine Learning Models:**
 - **Regression Models (Linear/Random Forest/XGBoost):** Predict ride counts using features like time, weather, events.
 - **Clustering + Forecasting:** Group zones with similar demand patterns, then forecast for each cluster.
-

5. Implementation Approach

1. **Zone Partitioning:** Divide city into grids/zones (using GPS coordinates or geohash).
 2. **Model Training:**
 - Train separate models per zone (or per cluster of zones).
 - Use historical + external features to predict demand.
 3. **Validation:**
 - Compare predicted vs. actual ride counts (using RMSE/MAE as metrics).
 - Fine-tune with cross-validation.
 4. **Deployment:**
 - Real-time predictions updated every few minutes using Spark Streaming or Kafka pipelines.
 - Predictions sent to driver apps for proactive positioning.
-

6. Expected Outcome

- Accurate **zone-wise demand forecasting**.
- Reduced passenger wait times.
- Better **driver allocation and route optimization**.
- Increased revenue and customer satisfaction.

A telecommunications company has been collecting data from its network operations...

(a) Explain the importance of data visualization in big data analytics and how it can help the telecommunications company solve its network congestion problem.

Answer:

Data visualization is the process of representing large and complex datasets in the form of visual formats such as graphs, charts, heat maps, and dashboards. In **big data analytics**, visualization plays a crucial role because:

1. **Simplifies Complex Data:** Telecom companies generate massive volumes of heterogeneous data (call records, internet usage, traffic logs). Visualization helps convert this raw data into meaningful patterns and trends.
2. **Pattern and Trend Identification:** Visual tools make it easier to spot recurring congestion during peak hours, heavy data usage zones, or sudden spikes in traffic.
3. **Quick Decision Making:** Managers and engineers can instantly understand where congestion occurs instead of analyzing raw numbers, allowing faster corrective action.
4. **Pinpointing Problem Areas:** Heat maps or geo-spatial visualizations can show areas of high network load, enabling the company to plan for additional bandwidth or infrastructure upgrades.
5. **Improved Customer Satisfaction:** By addressing congestion based on visual insights, the company can reduce dropped calls, slow internet speeds, and service complaints, leading to improved customer experience.

👉 Thus, data visualization bridges the gap between raw big data and actionable insights, directly helping the telecom company manage **network congestion** effectively.

(b) Discuss the different types of data visualizations (charts, graphs, dashboards, etc.) that would be suitable for analyzing the company's network traffic data. For each type, explain what insights the company can derive.

Answer:

Different visualization techniques provide different insights. Some suitable ones for telecom traffic analysis are:

1. Line Charts / Time-Series Graphs

- **Use:** To show internet usage, call traffic, or service requests over time (hourly, daily, weekly).
- **Insight:** Identifies **peak usage periods** and recurring congestion times.

2. Bar Charts / Column Charts

- **Use:** To compare usage across different regions, service types (voice, data, video), or customer segments.

- **Insight:** Reveals which **regions or services** face the highest load and need infrastructure investment.

3. Heat Maps / Geo-Spatial Maps

- **Use:** To visualize congestion intensity across different geographical areas.
- **Insight:** Pinpoints **specific zones** where congestion occurs frequently, aiding in tower placement or bandwidth distribution.

4. Pie Charts / Donut Charts

- **Use:** To show proportion of traffic sources (voice calls vs. data usage vs. service requests).
- **Insight:** Highlights **dominant traffic types** causing congestion.

5. Dashboards (Interactive)

- **Use:** Combine multiple visualizations (real-time traffic, alerts, error logs) in one interface.
- **Insight:** Provides a **comprehensive, real-time overview** of network health for quick monitoring and decisions.

6. Scatter Plots / Correlation Graphs

- **Use:** To analyze relationship between variables (e.g., internet speed vs. number of users).
- **Insight:** Helps detect whether congestion increases linearly with users or is influenced by other factors.

Final Summary for Exams:

- **Part (a):** Data visualization in big data helps simplify, detect patterns, and solve congestion by identifying recurring issues and enabling fast decisions.
- **Part (b):** Different visualization types (line, bar, heat maps, pie, dashboards, scatter plots) provide insights such as **peak times, heavy regions, dominant traffic sources, and real-time monitoring**, ultimately improving service quality and reducing congestion.

Given

User matrix P (3 users $\times$ 2 latent dims) and item matrix Q (4 items $\times$ 2 latent dims):

$$P = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \\ 0.3 & 0.7 \end{bmatrix}, \quad Q = \begin{bmatrix} 1 & 0.4 \\ 0.6 & 1 \\ 0.5 & 0.8 \\ 0.3 & 0.5 \end{bmatrix}.$$

Matrix-factorization predicts the user–item rating by the dot product of the user vector p_u and the item vector q_i :

$$\hat{r}_{u,i} = p_u \cdot q_i = p_{u,1}q_{i,1} + p_{u,2}q_{i,2}.$$

We compute only the **missing** entries (but you can verify any cell with the same formula).

-
- A User matrix P :

$$P = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \\ 0.3 & 0.7 \end{bmatrix}$$

This has **3 rows** (for 3 users), each row being the **latent feature vector** of a user.

- An Item matrix Q :

$$Q = \begin{bmatrix} 1 & 0.4 \\ 0.6 & 1 \\ 0.5 & 0.8 \\ 0.3 & 0.5 \end{bmatrix}$$

This has **4 rows** (for 4 movies), each row being the **latent feature vector** of a movie.

Step 2: Matching rows with users/items

- User vectors p_u come from rows of P .

- Row 1 → User 1
- Row 2 → User 2
- Row 3 → User 3

So,

$$p_1 = (1, 0.5), \quad p_2 = (0.5, 1), \quad p_3 = (0.3, 0.7)$$

- Item vectors q_i come from rows of Q .

- Row 1 → Movie 1
- Row 2 → Movie 2
- Row 3 → Movie 3
- Row 4 → Movie 4

So,

$$q_1 = (1, 0.4), \quad q_2 = (0.6, 1), \quad q_3 = (0.5, 0.8), \quad q_4 = (0.3, 0.5)$$

Calculations (step-by-step)

User 1, Movie 2:

$$p_1 = (1, 0.5), q_2 = (0.6, 1) \\ \hat{r}_{1,2} = 1 \cdot 0.6 + 0.5 \cdot 1 = 0.6 + 0.5 = \mathbf{1.10}.$$

User 2, Movie 1:

$$p_2 = (0.5, 1), q_1 = (1, 0.4) \\ \hat{r}_{2,1} = 0.5 \cdot 1 + 1 \cdot 0.4 = 0.5 + 0.4 = \mathbf{0.90}.$$

User 2, Movie 3:

$$p_2 = (0.5, 1), q_3 = (0.5, 0.8) \\ \hat{r}_{2,3} = 0.5 \cdot 0.5 + 1 \cdot 0.8 = 0.25 + 0.8 = \mathbf{1.05}.$$

User 3, Movie 2:

$$p_3 = (0.3, 0.7), q_2 = (0.6, 1) \\ \hat{r}_{3,2} = 0.3 \cdot 0.6 + 0.7 \cdot 1 = 0.18 + 0.7 = \mathbf{0.88}.$$

User 3, Movie 4:

$$p_3 = (0.3, 0.7), q_4 = (0.3, 0.5) \\ \hat{r}_{3,4} = 0.3 \cdot 0.3 + 0.7 \cdot 0.5 = 0.09 + 0.35 = \mathbf{0.44}.$$

(If you need, you can compute any other cell the same way; e.g. $\hat{r}_{1,1} = 1 \cdot 1 + 0.5 \cdot 0.4 = 1.2$.)

Final completed user–item matrix (filled missing entries)

User/Item	Movie 1	Movie 2	Movie 3	Movie 4
User 1	5	1.10	3	1
User 2	0.90	4	1.05	2
User 3	1	0.88	2	0.44

a) Describe the MapReduce paradigm (how Map and Reduce work)

High-level idea: MapReduce is a programming model for processing large datasets in parallel across a distributed cluster. The work is split into two main stages:

1. Map stage

- Input: a large dataset split into chunks (input splits).
- Each **Mapper** processes records in its split and emits intermediate key–value pairs: `(key, value)`.
- Example mapper task for this problem: read `(productID, saleAmount)` and `emit(productID, saleAmount)`.

2. Shuffle & Sort

- The system re-distributes intermediate pairs by key so that all values for the same key are sent to the same Reduce task.
- Usually includes sorting by key.

3. Reduce stage

- Each **Reducer** receives `(key, list_of_values)` and applies a reduce function (e.g., sum) to produce final `(key, outputValue)` pairs.
- For totals: `reduce(key, values) => emit(key, sum(values))`.

Key components & features

- **Partitioner:** decides which reducer gets which key (often hash-based).
- **Combiner (optional):** a local reducer run on mapper outputs to reduce network traffic (e.g., sum partial totals per mapper).
- **Fault tolerance:** failed mapper/reducer tasks can be retried.
- **Data locality:** tasks are scheduled where data resides when possible.

Pseudocode (simple):

Mapper pseudocode:

```
map(record):  
    // record = (productID, saleAmount)  
    emit(productID, saleAmount)
```

Reducer pseudocode:

```
reduce(key, values):  
    total = 0  
    for v in values:  
        total += v  
    emit(key, total)
```

b) Simulate Map and Reduce on the given sales data

Given data (ordered rows)

(From problem)

```
1. P1  100  
2. P2  200  
3. P1  150  
4. P3  250  
5. P2  300  
6. P3  100  
7. P1  200
```

Step 1 — Map phase (each record → intermediate (key, value))

If every mapper simply emits (product, amount), the raw intermediate pairs (in order) are:

(P1, 100)
(P2, 200)
(P1, 150)
(P3, 250)
(P2, 300)
(P3, 100)
(P1, 200)

(Those are the mapper outputs before any combining or grouping.)

Optional: Show Mappers & Combiners (typical practical optimization)

To illustrate parallelism and combiners, split the input among 3 mappers:

- **Mapper A** processes records 1–3:
 - emits: (P1,100), (P2,200), (P1,150)
 - **local combiner** on Mapper A sums P1 locally → (P1, 250), (P2, 200)
- **Mapper B** processes records 4–5:
 - emits: (P3,250), (P2,300)
 - local combiner → (P3,250), (P2,300)
- **Mapper C** processes records 6–7:
 - emits: (P3,100), (P1,200)
 - local combiner → (P3,100), (P1,200)

So after local combine, the intermediate pairs sent over the network (fewer pairs) are:

From Mapper A: (P1,250), (P2,200)
From Mapper B: (P3,250), (P2,300)
From Mapper C: (P3,100), (P1,200)

Step 2 — Shuffle & Grouping (all values for each product are collected)

Group by key:

- P1 → values: [250 (from A), 200 (from C)] → combined list [250, 200]
- P2 → values: [200 (from A), 300 (from B)] → combined list [200, 300]
- P3 → values: [250 (from B), 100 (from C)] → combined list [250, 100]

(If you had not used combiners, grouping would gather all raw mapper outputs into the same lists: P1 → [100,150,200], etc. The combiner simply reduced network traffic by partially summing earlier.)

Step 3 — Reduce phase (sum values per key)

Apply `reduce(key, values)` where reduce sums the list:

- **Reduce(P1, [250, 200])** → total = 250 + 200 = **450** → emit (P1, 450)
- **Reduce(P2, [200, 300])** → total = 200 + 300 = **500** → emit (P2, 500)
- **Reduce(P3, [250, 100])** → total = 250 + 100 = **350** → emit (P3, 350)

Final result after Reduce:

P1 450
P2 500
P3 350

(If you had not used combiners, the reducers would have summed the full lists: P1: 100+150+200 = 450, P2:200+300=500, P3:250+100=350 — same final result.)

c) Performance estimate (machines, records, times)

Given

- Each machine processes **10,000** sales records.
- There are **50 machines**.
- Map phase takes **0.02 seconds per record**.
- Reduce phase takes **0.05 seconds per key**.
- There are **1,000 unique products (keys)**.

Goal: estimate how long it will take to process the dataset.

Map phase time

- Each machine processes 10,000 records serially (per statement).
- Time per machine for map work = $10,000 \text{ records} \times 0.02 \text{ s/record} = 200 \text{ s}$.
- Map tasks are run in parallel on 50 machines, so **wall-clock time for Map stage** $\approx$ time of slowest mapper = **200 seconds** (assuming balanced load and no stragglers).

Shuffle & Sort time

- Not given explicitly. In a real cluster, shuffle (network transfer + sort) can add time; we will **ignore shuffle overhead** for this estimate unless asked to include it. (If you must include it, add a separate estimate based on network throughput.)

Reduce phase time

We must assume how reducers are allocated. Two reasonable scenarios:

(1) Best parallelism — one reducer per machine (50 reducers):

- Keys per reducer $\approx 1000 \text{ keys} / 50 \text{ reducers} = 20 \text{ keys/reducer}$.

- Time per reducer = $20 \text{ keys} \times 0.05 \text{ s/key} = 1.0 \text{ s}$.
- Reduce stage wall time $\approx$ max reducer time $\approx$ **1.0 second**.
- Total (Map + Reduce) $\approx 200 + 1 = 201 \text{ seconds} \approx$ **3 minutes 21 seconds**.

(2) Single reducer (worst for reduce parallelism):

- Single reducer must process all 1000 keys:
- Time = $1000 \times 0.05 \text{ s} = 50 \text{ s}$.
- Total $\approx 200 + 50 = 250 \text{ seconds} \approx$ **4 minutes 10 seconds**.

(3) Intermediate example — fewer reducers (e.g., 10 reducers):

- Keys per reducer = $1000 / 10 = 100$.
- Time/reducer = $100 \times 0.05 = 5 \text{ s}$.
- Total $\approx 200 + 5 = 205 \text{ s}$.

Final recommended estimate (state assumptions)

- If reducers $\approx$ number of machines (50) $\rightarrow$ **$\sim 201 \text{ s}$** (dominant cost is Map at 200 s).
- If only 1 reducer $\rightarrow$ **$\sim 250 \text{ s}$** .
- If 10 reducers $\rightarrow$ **$\sim 205 \text{ s}$** .

Important caveats to mention in exam:

- These numbers **ignore**: shuffle/network transfer overhead, serialization/deserialization, disk I/O, task startup time, straggler tasks and retries, and combining benefits. In practice the shuffle can add nontrivial time; combiner use reduces network traffic and can speed shuffle/reduce.
- Map stage is dominant here because each machine has many records; reducing number of records per machine or making mappers faster lowers wall time.

Aspect	Relational DBMS (RDBMS)	NoSQL Databases
Data Model	Structured data stored in tables (rows & columns).	Can be document-based, key-value, column-family, or graph-based (flexible schemas).
Schema	Fixed schema; altering structure requires schema migration.	Schema-less or dynamic schema; allows flexible, evolving data structures.
Scalability	Vertical scaling (adding more CPU/RAM to one machine).	Horizontal scaling (adding more machines/nodes in distributed systems).
Transactions	Strong ACID properties (Atomicity, Consistency, Isolation, Durability).	BASE model (Basically Available, Soft state, Eventual consistency) → trades strict consistency for scalability.
Query Language	SQL (standardized, powerful for joins and complex queries).	No standard query language; varies (e.g., MongoDB Query Language, Cassandra CQL).
Joins	Supports complex joins between multiple tables.	Generally does not support joins directly (data is denormalized for performance).
Use Cases	Banking, ERP, OLTP systems, where consistency and structured data are critical.	Big data, IoT, social networks, recommendation engines, real-time analytics, where scalability and flexibility are key.
Examples	Oracle, MySQL, PostgreSQL, SQL Server.	MongoDB, Cassandra, Redis, Neo4j, Couchbase.

Four Ways NoSQL Operates on Big Data Problems

NoSQL databases are designed to handle **Volume, Velocity, Variety, and Veracity** of big data. They operate in the following ways:

1. Distributed Storage & Horizontal Scalability

- Data is split across multiple servers/nodes (sharding & replication).

- This allows systems like Cassandra or MongoDB to scale linearly as data grows.

2. **Flexible Schema & Semi-Structured Data Handling**

- Supports JSON, XML, key-value pairs, wide columns, or graph models.
- Makes it easy to ingest heterogeneous and rapidly evolving big data.

3. **High-Speed Data Processing & Real-Time Access**

- Optimized for high write/read throughput (millions of operations per second).
- Useful in applications like clickstream analysis, IoT sensor feeds, and recommendation engines.

4. **Eventual Consistency & Fault Tolerance**

- Instead of strict ACID, NoSQL often uses **BASE principles**.
- Data is replicated across multiple nodes for high availability and resilience against failures.

Solution

Compute step-by-step (digit-by-digit):

Given: 100 million profiles = 100,000,000 profiles

Each profile $\approx$ 10 KB

1. Total storage (in KB):

$$100,000,000 \times 10 \text{ KB} = 1,000,000,000 \text{ KB.}$$

2. Storage per node (50 nodes):

$$1,000,000,000 \text{ KB} \div 50 = 20,000,000 \text{ KB per node.}$$

3. Convert to MB and GB (decimal, where 1 KB = 1,000 B, 1 MB = 1,000 KB, 1 GB = 1,000 MB):

$$20,000,000 \text{ KB} \div 1,000 = 20,000 \text{ MB}$$

$$20,000 \text{ MB} \div 1,000 = 20 \text{ GB.}$$

Answer (decimal units): 20 GB per node.

If you use binary units (1 KiB = 1024 bytes):

- Each doc = $10 \times 1024 = 10,240$ bytes.
- Total bytes = $100,000,000 \times 10,240 = 1,024,000,000,000$ bytes.
- Total GiB $\approx 1,024,000,000,000 \div 1024^3 \approx 953.6743$ GiB.
- Per node $\approx 953.6743 \div 50 \approx 19.0735$ GiB.

Binary answer: $\approx$ 19.07 GiB per node.

Note: the question doesn't mention replication or overhead. If a replication factor (e.g. 3 $\times$) is used, multiply these per-node numbers by the replication factor.

(a) Content-based recommender (cosine similarity)

Feature vectors:

- Book A = [1, 2, 0, 1]
- Book B = [2, 0, 1, 1]
- Book C = [0, 1, 2, 1]

Cosine similarity formula:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

(i) Cosine similarity between A and B

Dot product $A \cdot B$:

$$1 \cdot 2 + 2 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 = 2 + 0 + 0 + 1 = 3$$

Norms:

$$\|A\| = \sqrt{1^2 + 2^2 + 0^2 + 1^2} = \sqrt{1 + 4 + 0 + 1} = \sqrt{6}$$

$$\|B\| = \sqrt{2^2 + 0^2 + 1^2 + 1^2} = \sqrt{4 + 0 + 1 + 1} = \sqrt{6}$$

So

$$\cos(A, B) = \frac{3}{\sqrt{6} \sqrt{6}} = \frac{3}{6} = \frac{1}{2} = 0.5.$$

(ii) Which book to recommend to a user who likes Book A?

Compute cosine(A,C):

Dot product $A \cdot C$:

$$1 \cdot 0 + 2 \cdot 1 + 0 \cdot 2 + 1 \cdot 1 = 0 + 2 + 0 + 1 = 3$$

$$\|C\| = \sqrt{0^2 + 1^2 + 2^2 + 1^2} = \sqrt{0 + 1 + 4 + 1} = \sqrt{6}.$$

So

$$\cos(A, C) = \frac{3}{\sqrt{6} \sqrt{6}} = \frac{3}{6} = 0.5.$$

Both Book B and Book C have the same cosine similarity to A (0.5). Therefore you can recommend **either B or C (or both)**. If a single pick is required, use a tie breaker (e.g., popularity, user ratings, recency).



(b) SimRank — compute $S(X, Z)$ at iteration 1

SimRank formula (for nodes A, B):

$$S(A, B) = \frac{c}{|I(A)||I(B)|} \sum_{i \in I(A)} \sum_{j \in I(B)} S(i, j)$$

where $I(\cdot)$ = in-neighbours, and given $c = 0.8$.

Graph: nodes $\{X, Y, Z, W\}$ with edges $X \leftrightarrow Y$, $Y \leftrightarrow Z$, $Z \leftrightarrow W$, $W \leftrightarrow X$. So (undirected) neighbours:

- $I(X) = \{Y, W\}$
- $I(Z) = \{Y, W\}$

Thus $|I(X)| = 2$, $|I(Z)| = 2$.

We are told initial similarities (iteration 0): $S(i, i) = 1$ and $S(i, j) = 0$ for $i \neq j$.

Now compute $S(X, Z)$ at iteration 1:

$$S(X, Z) = \frac{c}{2 \cdot 2} (S(Y, Y) + S(Y, W) + S(W, Y) + S(W, W)).$$

Using initial values: $S(Y, Y) = 1$, $S(W, W) = 1$, $S(Y, W) = S(W, Y) = 0$. So the bracket sum = $1 + 0 + 0 + 1 = 2$.

Therefore

$$S(X, Z) = \frac{0.8}{4} \cdot 2 = \frac{0.8}{2} = 0.4.$$

Answer: $S(X, Z)$ at iteration 1 = **0.4**.

◆ 1. HDFS (Hadoop Distributed File System)

- **Type:** Distributed File System
- **Purpose:** Store *large volumes of unstructured or semi-structured data* reliably across many machines.
- **Data Model:** Stores files (no schema).
- **Access Pattern:** High-throughput, batch-oriented read/write (e.g., MapReduce jobs, Spark processing).

- **Use Cases:**
 - Big Data storage (logs, images, videos, sensor data)
 - Data lake for Hadoop ecosystem
 - Write once, read many times
-

◆ 2. HBase (Hadoop Database)

- **Type:** NoSQL column-oriented database (built on top of HDFS)
 - **Purpose:** Random, real-time read/write access to big data stored in HDFS.
 - **Data Model:**
 - Modeled after Google BigTable
 - Data stored in *tables* → *rows* → *column families* → *columns*
 - Schema-less (you can add new columns dynamically)
 - **Access Pattern:** Fast random read/write, good for sparse datasets.
 - **Use Cases:**
 - Real-time queries on large datasets
 - Storing time-series data, sensor data, logs
 - Backend for applications that need low-latency access to big data
-

◆ 3. RDBMS (Relational Database Management System)

- **Type:** Relational Database (e.g., MySQL, PostgreSQL, Oracle, SQL Server)
- **Purpose:** Store and manage *structured data* with relationships.
- **Data Model:** Strict schema → Tables, rows, columns, with constraints (primary key, foreign key).
- **Access Pattern:** ACID-compliant transactions, powerful SQL queries, optimized for consistency.
- **Use Cases:**
 - Banking systems, ERP, CRM
 - Applications where consistency and relationships matter (orders, customers, products)
 - OLTP (Online Transaction Processing)

◆ Key Differences (Tabular View)

Feature	HDFS	HBase	RDBMS
Type	Distributed file system	NoSQL (column-oriented DB)	Relational database
Data Model	Files (unstructured/semi-structured)	Tables (rows, column families, schema-less)	Tables (structured, schema-based)
Storage	Stores raw files in blocks	Stores data in HDFS tables	Stores structured records
Access Pattern	Batch, sequential read/write	Random, real-time read/write	Transactional (ACID), SQL queries
Schema	No schema	Flexible schema-less	Fixed schema
Latency	High latency (not for real-time)	Low latency (good for real-time)	Low latency for transactions
Use Case	Big Data storage (data lake)	Real-time analytics on big data	Business apps, OLTP systems
Scalability	Horizontal (add machines)	Horizontal (auto-sharding)	Vertical (limited by single machine)
ACID Compliance	No	No (eventual consistency)	Yes

◆ When to use what?

- **Use HDFS** → When you just need to *store huge files* and process them later (e.g., data lake, ML pipelines).
- **Use HBase** → When you need *real-time random access* to large datasets stored in Hadoop.
- **Use RDBMS** → When you need *transactional guarantees, structured schema, and relationships* between data.

Item-to-Item Similarity Example

Suppose we have **user ratings** for 4 movies:

User	Inception	Interstellar	Titanic	Avatar
U1	5	4	1	0
U2	4	5	2	1
U3	5	4	0	2
U4	0	1	5	4

Step 1: Choose target item

Let's calculate similarity of **Inception** with other movies.

Step 2: Compute similarity

We can use **Cosine Similarity** between item-rating vectors.

- Vector for *Inception* = [5, 4, 5, 0]
- Vector for *Interstellar* = [4, 5, 4, 1]
- Vector for *Titanic* = [1, 2, 0, 5]
- Vector for *Avatar* = [0, 1, 2, 4]

Step 3: Cosine similarity formula

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

(a) Inception vs Interstellar

- Dot product = $(5 \times 4 + 4 \times 5 + 5 \times 4 + 0 \times 1) = 20 + 20 + 20 + 0 = 60$
 - $\| \text{Inception} \| = \sqrt{(5^2 + 4^2 + 5^2 + 0^2)} = \sqrt{(25 + 16 + 25)} = \sqrt{66} \approx 8.12$
 - $\| \text{Interstellar} \| = \sqrt{(4^2 + 5^2 + 4^2 + 1^2)} = \sqrt{(16 + 25 + 16 + 1)} = \sqrt{58} \approx 7.62$
 - Similarity = $60 / (8.12 \times 7.62) \approx 0.97$  (Very similar)
-

(b) Inception vs Titanic

- Dot product = $(5 \times 1 + 4 \times 2 + 5 \times 0 + 0 \times 5) = 5 + 8 + 0 + 0 = 13$
 - $\| \text{Titanic} \| = \sqrt{(1^2 + 2^2 + 0^2 + 5^2)} = \sqrt{(1 + 4 + 25)} = \sqrt{30} \approx 5.48$
 - Similarity = $13 / (8.12 \times 5.48) \approx 0.29$ (Low similarity)
-

(c) Inception vs Avatar

- Dot product = $(5 \times 0 + 4 \times 1 + 5 \times 2 + 0 \times 4) = 0 + 4 + 10 + 0 = 14$
- $\| \text{Avatar} \| = \sqrt{(0^2 + 1^2 + 2^2 + 4^2)} = \sqrt{(1 + 4 + 16)} = \sqrt{21} \approx 4.58$
- Similarity = $14 / (8.12 \times 4.58) \approx 0.37$ (Moderate similarity)

Step 4: Recommendation

- **Most similar item to Inception = Interstellar (0.97)**
- Titanic (0.29) and Avatar (0.37) are much less similar.

Q1. Explain Association Rules with an example.

Answer:

- **Definition:** Association rules are “if-then” statements used to discover interesting relationships between variables in large datasets.

- **Form:**

- Rule: $X \Rightarrow Y$
- X : Antecedent (if part), Y : Consequent (then part)

- **Key Measures:**

1. **Support:** Probability that both X and Y occur together.

$$\text{Support}(X \Rightarrow Y) = \frac{\text{Transactions containing } (X \cup Y)}{\text{Total Transactions}}$$

2. **Confidence:** Likelihood of Y occurring given X.

$$\text{Confidence}(X \Rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

3. **Lift:** Strength of rule compared to random chance.

$$\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)}$$

- **Example:**

Dataset of supermarket transactions:

- T1: {Milk, Bread}
 - T2: {Milk, Diaper, Beer, Bread}
 - T3: {Milk, Diaper, Beer, Cola}
 - T4: {Bread, Milk, Diaper, Beer}
 - T5: {Bread, Milk, Cola}
- Rule: {Milk, Diaper} $\Rightarrow$ {Beer}
 - Support = $3/5 = 60\%$
 - Confidence = $3/3 = 100\%$
 - Lift > 1 (strong rule)

✓ This shows customers buying Milk & Diaper are likely to buy Beer.

Q2. Describe the Apriori Algorithm with an example.

Answer:

- **Purpose:** Finds frequent itemsets and generates association rules.
- **Steps:**
 - Scan dataset → find frequent items with minimum support.
 - Generate candidate itemsets (k+1) from frequent k-itemsets.
 - Prune itemsets that don't meet support threshold.
 - Repeat until no more itemsets can be generated.
- **Example:**

For the above supermarket data, if minimum support = 60%, Apriori will find frequent sets:

 - {Milk}, {Bread}, {Beer}, {Diaper}
 - {Milk, Bread}, {Milk, Diaper}, {Diaper, Beer}
 - From these, rules like {Milk, Diaper} ⇒ {Beer} can be generated.

Q3. How do we evaluate candidate rules? Explain with metrics.

Answer:

Candidate rules are evaluated using:

1. **Support** – Frequency of rule occurrence.
 2. **Confidence** – Reliability of rule.
 3. **Lift** – Measures dependency between X and Y.
 4. **Conviction** – Likelihood that X occurs without Y.
- **Example:** Rule {Bread} ⇒ {Milk}
 - Support = $4/5 = 80\%$

- Confidence = $4/4 = 100\%$
- Lift = $(1.0)/(0.8) = 1.25 \rightarrow$ Strong positive correlation

Q4. What are applications of Association Rules?

Answer:

1. **Market Basket Analysis** – Identifying products often bought together (e.g., “Milk & Bread”).
2. **Cross-selling & Upselling** – Amazon “Frequently Bought Together.”
3. **Fraud Detection** – Identifying unusual patterns in banking transactions.
4. **Healthcare** – Discovering symptoms associated with diseases.
5. **Web Usage Mining** – Analyzing clickstream data to suggest relevant pages.

Q5. Explain Collaborative Recommendation with example.

Answer:

- **Definition:** Suggests items to a user based on preferences of similar users.
- **Types:**
 1. **User-based:** Finds users with similar tastes.
 2. **Item-based:** Finds items liked by similar users.
- **Example:**
Movie ratings table:

User	Movie A	Movie B	Movie C
U1	5	3	?
U2	4	2	5
U3	5	3	4

- To predict U1's rating for Movie C, system compares U1 with U2 & U3 (similar users) → predicts rating close to 4–5.

✓ Widely used in **Netflix, Spotify**.

Q6. Explain Content-Based Recommendation with example.

Answer:

- **Definition:** Recommends items similar to those the user has liked before, based on item features.
- **Process:**
 1. Analyze features of items (genre, keywords, description).
 2. Build user profile from liked items.
 3. Recommend new items matching profile.
- **Example:**

If a user liked *Inception* (Sci-Fi, Thriller), the system recommends *Interstellar* (Sci-Fi, Space).

✓ Used in **news websites, YouTube**.

Q7. Explain Knowledge-Based and Hybrid Recommendation Systems.

Answer:

1. **Knowledge-Based Recommendation**
 - Works on explicit knowledge (rules, constraints, expert knowledge).
 - Example: Travel site suggests destinations based on “Budget < ₹20,000” & “Beach Preference = Yes.”

2. Hybrid Recommendation

- Combines collaborative + content-based + knowledge-based methods.
- Example: Netflix uses:
 - **Content-based** for analyzing movies' features,
 - **Collaborative** for comparing with similar users,
 - **Hybrid** final recommendation.

Q8. Compare Collaborative, Content-Based, and Hybrid Approaches.

Answer:

Feature	Collaborative Filtering	Content-Based Filtering	Hybrid System
Data Requirement	User-item ratings	Item features	Both
Strength	Captures community trends	Personalized to user	More accurate
Weakness	Cold-start problem, sparse data	Limited novelty, overspecialization	Complex to implement
Example	Netflix, Amazon reviews	Spotify genre-based recs	Netflix final system

Clustering: Definition

Clustering is an **unsupervised machine learning technique** used to **group a set of objects into clusters** such that:

- Objects in the **same cluster are similar** to each other.
- Objects in **different clusters are dissimilar**.

It is widely used in **data mining, pattern recognition, image segmentation, market segmentation, and recommendation systems**.

Example:

Suppose a shopping website wants to segment customers based on purchasing behavior. Customers who buy similar products frequently will be grouped into one cluster, while those with different buying patterns go into another cluster.

Types of Clustering

Clustering can be classified into several types based on methodology. The major types are:

1. Partitioning Clustering

- **Definition:** Divides the dataset into **non-overlapping subsets** (clusters) where each object belongs to exactly one cluster.
- **How it works:** Starts with an initial set of cluster centers, assigns points to the nearest cluster, then updates cluster centers iteratively.
- **Common Algorithms:** K-Means, K-Medoids

Example:

Suppose we have customer data with attributes like **age** and **income**. Using K-Means with $k=3$, we might get three clusters:

1. Young & low income
2. Middle-aged & medium income
3. Senior & high income

Each customer is assigned to exactly one cluster.

2. Hierarchical Clustering

- **Definition:** Creates a **tree-like structure** (dendrogram) representing nested clusters.
- **Types:**

- **Agglomerative (Bottom-Up):** Each data point starts as its own cluster; clusters are merged step by step.
- **Divisive (Top-Down):** The whole dataset starts as a single cluster and is recursively split.
- **Example:**
A dendrogram of animals based on features (legs, habitat, diet) might show:
 - Cluster 1: Mammals (dogs, cats)
 - Cluster 2: Birds (sparrow, eagle)
 - Cluster 3: Fish (salmon, tuna)

Advantages: No need to predefine the number of clusters.

3. Density-Based Clustering

- **Definition:** Forms clusters based on **dense regions** of points, separating sparse regions as noise.
- **Key Idea:** Clusters are areas where the density of points exceeds a threshold.
- **Popular Algorithm:** DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Example:

In a GPS dataset of city taxis:

- Dense regions represent areas where many taxis are concentrated (downtown)
- Sparse regions represent highways or suburbs (noise)

Advantage: Can find arbitrarily shaped clusters and is robust to noise.

4. Grid-Based Clustering

- **Definition:** Divides the data space into a **finite number of cells (grid)** and performs clustering on these grids.
- **How it works:**
 1. Quantize the data space into a grid structure.
 2. Find dense cells and form clusters from adjacent dense cells.
- **Popular Algorithm:** STING (Statistical Information Grid)

Example:

In satellite imaging, pixels in similar regions of a landscape can form clusters representing **forest, water, and urban areas**.

5. Model-Based Clustering

- **Definition:** Assumes data is generated by a **mixture of underlying probability distributions**.
- **How it works:**
 - Fits the data to a model (e.g., Gaussian Mixture Model).
 - Assigns points to clusters based on likelihood.
- **Example:**

For exam scores of students:

 - One cluster could represent "high achievers" (Gaussian with high mean)
 - Another cluster "average" (Gaussian with medium mean)
 - Another cluster "low" (Gaussian with low mean)

Advantage: Can handle complex cluster shapes.

6. Fuzzy Clustering

- **Definition:** Assigns **degrees of membership** to each data point rather than a single cluster.
 - **Algorithm:** Fuzzy C-Means
 - **Example:**
A movie recommendation system: a user may belong 70% to "action lover" cluster and 30% to "romantic lover" cluster, reflecting mixed interests.
-

Comparison Table of Clustering Types

Type	Key Concept	Example	Advantage	Limitation
Partitioning	Assign to nearest cluster	K-Means on customer data	Simple, fast	Needs k in advance
Hierarchical	Tree structure	Animal classification	Dendrogram shows relationships	Slow for large data
Density-Based	Dense regions form clusters	GPS taxi data	Finds arbitrary shapes, handles noise	Sensitive to parameters
Grid-Based	Divide space into grids	Satellite images	Fast for large data	Resolution dependent
Model-Based	Fit statistical model	Student scores	Handles complex clusters	Model assumption required
Fuzzy	Membership in multiple clusters	Movie preferences	Reflects overlapping clusters	Computationally intensive

Decision Tree in R Programming

A **decision tree** is a flowchart-like model where each internal node represents a decision based on a feature, each branch represents an outcome of that decision, and each leaf node represents a final prediction. The algorithm recursively splits the data into subsets based on feature values to maximize homogeneity in the resulting groups.

Key concepts include:

- 1. **Root Node:** The topmost decision node.
- 2. **Splitting Criteria:** Metrics like **Gini impurity** (classification) or **variance reduction** (regression) determine how to split data.
- 3. **Pruning:** Techniques to trim branches that overfit the training data.
- 4. **Leaf Nodes:** Terminal nodes providing final predictions.

Summary Table

Step	R Function	Purpose
Build Tree	<code>rpart()</code>	Create decision tree
Visualize	<code>rpart.plot()</code>	Plot the tree
Predict	<code>predict()</code>	Predict new observations
Prune	<code>prune()</code>	Simplify tree to avoid overfitting

1. `rpart()` – Build the Decision Tree

The `rpart()` function (Recursive Partitioning and Regression Trees) is used to **create a decision tree**.

Syntax:

R

 Copy code

```
rpart(formula, data, method, control)
```

Parameters:

Parameter	Description
<code>formula</code>	Specifies the target variable and predictors. Format: <code>Target ~ Feature1 + Feature2 + ...</code> or <code>Target ~ .</code> to use all features.
<code>data</code>	The dataset used to train the tree.
<code>method</code>	<code>"class"</code> for classification, <code>"anova"</code> for regression, <code>"exp"</code> for survival analysis.
<code>control</code>	A list of control parameters, e.g., <code>minsplit</code> , <code>maxdepth</code> , <code>cp</code> .

Important Control Parameters:

- `minsplit`: Minimum observations required to attempt a split (default 20).
- `maxdepth`: Maximum depth of the tree (default 30).
- `cp` (complexity parameter): Minimum improvement in fit required to make a split (default 0.01).

2. `rpart.plot()` – Visualize the Tree

The `rpart.plot()` function is used to draw a graphical representation of the tree.

Syntax:

R

 Copy code

```
rpart.plot(x, type, extra, fallen.leaves, box.palette)
```

Parameters:

Parameter	Description
<code>x</code>	The decision tree object created by <code>rpart()</code> .
<code>type</code>	Type of plot: 1, 2, 3, 4, etc. (controls node labels). Type 2 is most common.
<code>extra</code>	Adds extra info: 0 → none, 1 → # of observations, 104 → class probabilities and n obs.
<code>fallen.leaves</code>	If TRUE, positions leaf nodes at the bottom for easier reading.
<code>box.palette</code>	Optional color palette for nodes.

Example:

R

 Copy code

```
library(rpart.plot)  
rpart.plot(iris_tree, type = 2, extra = 104, fallen.leaves = TRUE)
```

Interpretation:

- Rectangles show nodes.
- Text inside shows split condition, class probabilities, and number of observations.
- Leaf nodes show predicted class.



3. `predict()` – Make Predictions

The `predict()` function is used to **predict outcomes** from a trained decision tree.

Syntax:

R

 Copy code

```
predict(object, newdata, type)
```

Parameters:

Parameter	Description
<code>object</code>	The trained <code>rpart</code> tree object.
<code>newdata</code>	Data frame of new observations to predict.
<code>type</code>	<code>"class"</code> → predicted class labels, <code>"prob"</code> → class probabilities.

Example:

R

 Copy code

```
# Predict class labels
pred_class <- predict(iris_tree, iris, type = "class")

# Predict probabilities
pred_prob <- predict(iris_tree, iris, type = "prob")
```

- `pred_class` gives "setosa", "versicolor", or "virginica".
- `pred_prob` gives a table like `[0.99, 0.01, 0.00]` showing probability for each class.

4. `prune()` – Reduce Overfitting

Pruning **removes nodes** that contribute little predictive power to avoid overfitting.

Syntax:

R

 Copy code

```
prune(tree, cp)
```

Parameters:

Parameter	Description
<code>tree</code>	The <code>rpart</code> tree object.
<code>cp</code>	Complexity parameter value. Nodes with improvement below this value are removed.

Steps to Prune:

1. Check complexity parameter table:

R

 Copy code

```
printcp(iris_tree)
```

2. Find optimal `cp` (usually the one minimizing xerror).
3. Prune the tree:

R

 Copy code

```
pruned_tree <- prune(iris_tree, cp = 0.02)  
rpart.plot(pruned_tree, type = 2, extra = 104)
```

- Pruned tree is **simpler and less likely to overfit**, while retaining predictive accuracy.

Building Decision Trees in R

Let's build a decision tree to classify iris flowers into species (setosa, versicolor, or virginica).

1. Load Data and Split into Train/Test Sets

Before building a decision tree, we need to load the dataset and prepare it for training.

```
library(rpart)
library(rpart.plot)
library(caret)

data(iris)
set.seed(123)
train_index <- createDataPartition(iris$Species, p = 0.8, list = FALSE)
train_data <- iris[train_index, ]
test_data <- iris[-train_index, ]
```

2. Train the Model

Once the data is ready, we can build the Decision Tree model.

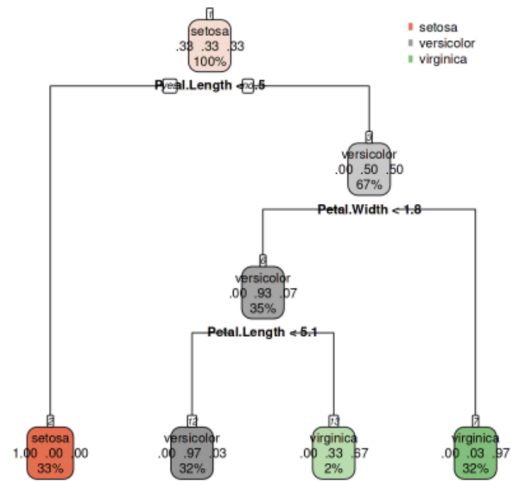
```
tree_model <- rpart(Species ~ .,
                    data = train_data,
                    method = "class", # For classification
                    control = rpart.control(minsplit = 10, cp = 0.01))
```

The model is trained using the Gini Index or Entropy to determine the best splits.

3. Visualize the Tree

```
rpart.plot(tree_model, box.palette = "auto", nn = TRUE)
```

Output



Decision Tree

This generates a tree diagram showing decision rules (e.g., petal length ≤ 2.5 for setosa). The tree uses Petal Length and Petal Width to classify flowers into three species: Setosa, Versicolor, and Virginica. The structure visually explains how the classification happens based on different feature thresholds.

4. Make predictions and Evaluate

To check model performance, we predict species for the test dataset.

```
predictions <- predict(tree_model, test_data, type = "class")
confusionMatrix(predictions, test_data$Species)
```

Output:

```

              Reference
Prediction setosa versicolor virginica
setosa      10         0         0
versicolor  0         10        1
virginica   0         0         9

Overall Statistics

      Accuracy : 0.9667
    95% CI : (0.8278, 0.9992)
  No Information Rate : 0.3333
    P-Value [Acc > NIR] : 2.963e-13

      Kappa : 0.95

McNemar's Test P-Value : NA

Statistics by Class:

              Class: setosa Class: versicolor Class: virginica
Sensitivity              1.0000              1.0000              0.9000
Specificity              1.0000              0.9500              1.0000
Pos Pred Value           1.0000              0.9091              1.0000
Neg Pred Value           1.0000              1.0000              0.9524
Prevalence                0.3333              0.3333              0.3333
Detection Rate           0.3333              0.3333              0.3000
Detection Prevalence     0.3333              0.3667              0.3000
Balanced Accuracy         1.0000              0.9750              0.9500
```

The confusion matrix shows accuracy, precision, and recall. The accuracy is high for all three species.

Decision Trees are easy to interpret and visualize. We can fine-tune it further by adjusting hyperparameters like pruning.

5. Prune the Tree

Avoid overfitting by pruning using the complexity parameter (cp):

```
printcp(tree_model) # Identify optimal cp (xerror is minimized)
optimal_cp <- tree_model$cpstable[which.min(tree_model$cpstable[, "xerror"]), "CP"]
pruned_tree <- prune(tree_model, cp = optimal_cp)
rpart.plot(pruned_tree)
```

Output:

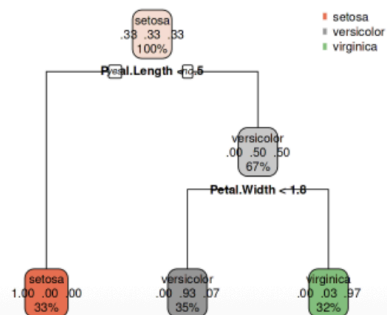
```
Classification tree:
rpart(formula = Species ~ ., data = train_data, method = "class",
      control = rpart.control(minsplit = 10, cp = 0.01))

Variables actually used in tree construction:
[1] Petal.Length Petal.Width

Root node error: 80/120 = 0.66667

n= 120

      CP nsplit rel error xerror      xstd
1 0.5000      0   1.0000 1.1375 0.058619
2 0.4500      1   0.5000 0.6375 0.067691
3 0.0125      2   0.0500 0.1000 0.034157
4 0.0100      3   0.0375 0.1125 0.036066
```



UNIT-5 STREAM MEMORY

1. Introduction to Streams Concepts

Definition

A **data stream** is a real-time, continuous, rapid, time-varying sequence of data items that arrive in an unbounded fashion.

Unlike static datasets (stored in databases), streams **cannot be fully stored**, and algorithms must process them **online** using limited memory.

Characteristics of Data Streams

1. **Unbounded** – data keeps arriving without end.
2. **Real-time / Near-real-time** – low latency processing is required.
3. **High velocity** – incoming rate may be extremely high (Gbps, millions of events/sec).
4. **One-pass constraint** – data can be seen **once** or very few times.
5. **Limited memory** – algorithms must use *sublinear memory* ($O(\log n)$, $O(1)$, etc.).
6. **Approximate Answers** – exact computation may be impossible; streaming algorithms provide approximations with error guarantees.

Examples

- Network traffic logs
- Sensor readings (IoT)
- Social media feeds (tweets, comments)
- Financial transactions
- Click-stream data from websites
- Real-time video analytics metadata

Why Data Streams?

- Traditional DBMS cannot handle velocity + volume.
 - Need real-time decision-making (fraud detection, anomaly detection, sentiment analysis).
 - Storage cost is too high to store raw streams.
-

2. Stream Data Model and Architecture

Stream Data Model

A formal representation of data streams that supports:

- **Sequential arrival** of tuples
- **Time-stamps** to maintain ordering
- **Windows** (finite subsets used for computations)

General Model

A stream S is:

$S = \langle (x_1, t_1), (x_2, t_2), (x_3, t_3), \dots \rangle$

where x_i = data item, t_i = time of arrival.

Types of Windows

1. **Tumbling Window** – fixed size, non-overlapping
2. **Sliding Window** – fixed size, overlapping
3. **Session Window** – based on inactivity gap
4. **Count-based Window** – based on number of items
5. **Time-based Window** – based on time duration

Stream Processing Architecture

Typical Architecture Blocks

1. **Data Sources**
Sensors, APIs, social media, logs, IoT.

2. Ingestion Layer

Tools like Kafka, Flume push data into system.

3. Stream Processing Layer

Real-time engines: Spark Streaming, Flink, Storm, Samza.

Tasks: filtering, sampling, aggregation, join, anomaly detection.

4. Storage Layer

- Hot storage: Redis, Memcached
- Cold storage: HDFS, cloud, DBs

5. Serving Layer

Dashboards, ML models, alert systems.

Architecture Diagram (ASCII)

```
Data Sources → Ingestion (Kafka) → Stream Processor (Flink/Spark)
                → Hot Storage (Redis) → Serving Layer (Dashboard)
                → Cold Storage (HDFS) → Batch Analytics
```

Functional Requirements

- Low latency
 - Fault tolerance
 - Scalability
 - Exactly-once / at-least-once guarantees
 - Event-time vs processing-time handling
-

3. Stream Computing

Definition

Stream computing refers to real-time computation on continuous data streams with minimal latency, often using **in-memory processing** and **parallel distributed engines**.

Goals

- Process events as soon as they occur
- Scale horizontally
- Provide approximate results with deterministic error bounds
- Handle data skew and bursty traffic

Types

1. **Complex Event Processing (CEP)** – identifies patterns (e.g., fraud detection).
2. **Continuous Query Processing** – SQL-like queries on streams.
3. **Real-time Analytics** – dashboards, KPIs.
4. **Streaming Machine Learning** – online learning algorithms.

Challenges

- Ordering out-of-order events
- Distributed consistency
- Maintaining state with limited memory
- Fault recovery in real-time

Examples of Stream Computing Systems

- Apache Storm
 - Apache Flink
 - Apache Spark Streaming
 - Google Dataflow
 - AWS Kinesis
-

4. Sampling Data in a Stream

When data is too large to store, we store only a **sample**.

Streaming sampling ensures a representative sample using $O(1)$ or $O(\log n)$ memory.

Why Sampling?

- Full archive is impossible
 - Fast approximate analytics
 - Used in ML, stock analysis, and real-time dashboards
-

4.1 Reservoir Sampling

Goal

Select **k** random items from a large or infinite stream with **equal probability**.

Algorithm (Reservoir Sampling for $k = 1$)

1. Take first item → store as sample.

2. For i -th item ($i > 1$):
 - Replace current sample with probability $1/i$.

Algorithm for general k

1. Store first k items in reservoir R .
2. For $i > k$:
 - Generate random number r between 1 and i .
 - If $r \leq k$, replace $R[r]$ with the new item.

Proof of fairness

Each item appears in final sample with probability k/n .

Time Complexity

- Time: $O(n)$
- Memory: $O(k)$

Use Cases

- Random tweet sampling
- Selecting random users
- Monitoring sensor streams
- Online A/B testing

4.2 Priority Sampling (Weighted Sampling)

Goal

Choose items proportionally to weight.

Technique

Assign each item weight w and priority $p = w / \text{random}(0,1)$.

Maintain top-k priorities.

Use Cases

- Stock selection weighted by market cap
 - Network packet sampling proportional to size
-

4.3 Sliding Window Sampling

Only sample most recent **W** items.

Methods:

- Biased reservoir
 - Exponential histograms
 - Windowed sampling with decay
-

5. Filtering Streams

Filtering selects only relevant data from continuous streams.

Definition

A stream filtering algorithm processes incoming items and keeps only those that satisfy certain **conditions/predicates**.

Examples:

- Keep only packets from suspicious IPs
- Keep tweets containing "crypto"
- Keep sensor readings above threshold
- Remove invalid or duplicate data

Categories of Filtering

1. Deterministic Filtering

- Based on fixed rules
- Example: if (value > 100) accept item

2. Probabilistic Filtering

- Uses sketching or hashing
 - Bloom filters
 - Count-Min Sketch
-

5.1 Bloom Filter

Definition

A Bloom Filter is a **probabilistic**, **space-efficient** structure that supports:

- **Membership queries**: "Is x in the set?"
- Allows **false positives**, no false negatives.

Structure

- Bit array of size m

- k independent hash functions

Operations

1. **Insert(x)**
Set all hash positions to 1
2. **Query(x)**
If all k positions are 1 $\rightarrow$ *possibly in set*
Else $\rightarrow$ *definitely not in set*

Memory

$O(m)$

Use Cases

- Duplicate detection in streams
- Web crawlers (visited URLs)
- Packet filtering in routers
- Database query optimization

5.2 Count-Min Sketch for Frequency Filtering

Definition

A CMS is an array of $d \times w$ counters updated with multiple hash functions.

Function

For each incoming element x:

- Update counter[h_i(x), i]++

To estimate frequency:

$$\text{freq}(x) = \min \text{ over } i [\text{CMS}[i][h_i(x)]]$$

Use Cases

- Detect heavy hitters
- Filter low-frequency words
- Network anomaly detection

1. COUNTING DISTINCT ELEMENTS IN A STREAM

1. Concept & Motivation

Counting distinct elements means estimating:

F_0 = number of unique items in the stream.

Challenges:

- Streams can be extremely large (billions of events).
- Cannot store all items (memory too small).
- Need approximate counting with small error and probabilistic guarantees.

Streaming distinct counting leads to **probabilistic sketches** such as:

- Flajolet–Martin (FM)
 - LogLog
 - HyperLogLog (HLL) — industry standard (used in Redis, BigQuery, Spark).
-

2. Theoretical Basis

2.1 Hashing to Uniform Bit-strings

Idea:

- Map each incoming item x to a uniformly random binary string $h(x)$ using a universal hash function.
- Binary strings behave like random coin toss sequences.

2.2 Core Observation

For a truly random stream:

- With more distinct elements, you are more likely to see a hash that has **many leading zeros**.

Example:

Probability that a random binary hash begins with k zeros:

$$P(\text{prefix of } k \text{ zeros}) = 1 / (2^k)$$

Thus:

- If the maximum number of leading zeros observed = R , then approximately:

$$\text{Distinct count} \approx 2^R$$

3. Flajolet–Martin (FM) Algorithm

Algorithm summary:

1. For each item x , compute $h(x)$.

Find the position of first 1-bit in $h(x)$:

$001011\dots \Rightarrow R(x) = 3$

- 2.

Track:

$R_{\max} = \max R(x) \text{ over entire stream}$

- 3.

Estimate:

$F0_estimate \approx 2^{R_{\max}}$

- 4.

Memory:

$O(1)$ (just one integer)

Problem:

High variance $\rightarrow$ solved by parallel FM sketches $\rightarrow$ LogLog $\rightarrow$ HLL.

4. HyperLogLog (HLL) — Practical Standard

Uses:

- Redis COUNT DISTINCT
- Google BigQuery's APPROX_COUNT_DISTINCT
- Spark, Presto

Core improvements:

- Splits stream into **m registers** (typically $m = 2^p$).
- For each item x:
 - Use first p bits of hash to select a register.
 - Use remaining bits to compute number of leading zeros.

Maintain:

```
register[j] = max(register[j], R(x))
```

-

Final estimate:

```
E = α_m * m^2 / ( Σ_j 2^(-register[j]) )
```

Where:

- α_m is a correction constant for bias removal.

Complexity:

- Time: $O(1)$ per item
- Memory: $O(m)$ (logarithmic in desired relative error)

Accuracy:

Relative error $\approx 1.04 / \sqrt{m}$

2) ESTIMATING MOMENTS (F_k)

1. Definition

Given frequencies f_i of each distinct item i , the k -th frequency moment is:

$$F_k = \sum_i (f_i^k)$$

Special cases:

F_0 = number of distinct elements

F_1 = stream length

F_2 = sum of squares of frequencies

F_k for $k > 2$ captures skew and heavy hitters

2. Why moments matter

- **F2** is crucial for:
 - detecting heavy hitters

- measuring similarity
 - variance estimation
 - **Higher moments** detect skew, bursts, anomalies.
-

3. Streaming Constraints

We cannot maintain all f_i exactly due to memory constraints.

Goal:

Approximate F_k in sublinear space (polylogarithmic memory).

4. Alon–Matias–Szegedy (AMS) Sketch

Classic algorithm for F_2 and general F_k .

Core idea (for F_2):

Define a random sign function:

$g(i) = +1$ or -1 (each with probability $1/2$)

Maintain:

$Z = \sum \text{over stream items } x \text{ of } g(x)$

Then:

$E[Z^2] = F_2$

Generalization for F_k :

More complex polynomial estimators; same principle: design a random variable whose expectation equals F_k .

Implementation features:

- Use multiple independent estimators $\rightarrow$ average $\rightarrow$ reduce variance.
 - Memory: $O((1/\epsilon^2) \log(1/\delta))$
-

5. Count-Sketch for F_2

Another method:

Maintain d hash functions, each mapping items to buckets with signs.

Estimator sketch:

For each row r :

$$\text{Est}_r = \sum_b (\text{count}[r][b]^2)$$

Output median over r

Complexity:

- Time per element: $O(d)$
 - Memory: $O((1/\epsilon) \log(1/\delta))$
-

3) COUNTING ONES IN A WINDOW (SLIDING WINDOW COUNTING)

1. Problem Definition

Binary stream:

... 1 0 1 1 0 0 1 ...

Goal:

At any time t , count number of 1s in last W items.

Challenges:

- Cannot store full window if W is large.
 - Need compact, update-efficient representation.
-

2. Exact Method — Queue / Deque (simple)

Maintain a queue of last W bits.

Implementation:

- Append new bit.
- If queue size $> W$: remove oldest.

Complexity:

- Time: $O(1)$

- Memory: $O(W)$

Not feasible for large $W \rightarrow$ Use approximate methods.

3. Approximate Counting — Exponential Histograms (EH)

Most widely used algorithm for sliding-window aggregates.

Key idea:

Summaries called *buckets*:

- Each bucket contains:
 - approximate count of 1s
 - timestamp of oldest bit
- Buckets are maintained so that:
 - $\text{size}(\text{Bucket}_i) = 2^i$ or $2^i \pm 1$
 - at most 2 buckets of the same size exist

Example ASCII diagram:

```
Window -----> |B1|B1|B2|B2|B4|
Sizes:           1  1  2  2  4
```

Algorithm:

When a new bit = 1:

- Create a size-1 bucket.

Merge buckets to maintain:

≤ 2 buckets of any size

-

Counting ones in window:

- Sum all bucket sizes except the oldest bucket (partially counted).

Error bound:

Relative error $\leq 1 / W$

Memory:

$O(\log^2 W)$

4) DECAYING WINDOW

1. Motivation

Sometimes recent data should matter more than older data.
Instead of a hard sliding window, apply smooth decay.

Examples:

- Real-time trending topics
 - Fraud detection
 - CPU load averages (OS uses exponential decay)
-

2. Formal Definition

Given stream of values v_t arriving at time t .

Define **exponential decay factor**:

$$0 < \lambda < 1$$

Weight for an item older by Δt time units:

$$\text{weight} = \lambda^{(\Delta t)}$$

Decayed sum at time T :

$$S(T) = \sum_t (v_t * \lambda^{(T - t)})$$

This ensures:

- Very old events contribute almost zero.
- No need to store complete window.

3. Online Update Formula (critical)

Let $S(T)$ be decayed sum at time T .

When new value v arrives at time $T+1$:

$$S(T+1) = \lambda * S(T) + v$$

This is highly efficient:

- One multiplication + one addition per event.

- Constant memory.
-

4. Applications

Streaming averages:

$$\text{avg} = S(T) / (1 - \lambda^{\text{window_size}})$$

- - EWMA (Exponentially Weighted Moving Average)
 - Trend detection
 - Load balancing
 - Sensor analytics
 - Stock tick smoothing
-

1. REAL-TIME ANALYTICS PLATFORM (RTAP) APPLICATIONS

1. Definition — What is an RTAP?

A **Real-Time Analytics Platform (RTAP)** is a computing environment designed to ingest, process, analyze, and act on **continuous data streams** with **low latency**, typically from milliseconds to a few seconds.

Key characteristics:

- **Unbounded data streams** → continuous arrival of events.
- **Low-latency processing** → decisions or outputs must be near real time.
- **Stateful computation** → system maintains in-memory state over windows.

- **Fault tolerance** → recovery via checkpoints, logs, replay mechanisms.
- **Scalable distributed design** → parallel workers processing partitioned streams.
- **Event-time correctness** → handles late/out-of-order events.

Examples of RTAP:

Kafka + Flink, Spark Structured Streaming, Google Dataflow, Amazon Kinesis Data Analytics, Apache Storm.

2. Why RTAP? (Conceptual Importance)

Traditional batch processing (MapReduce) is insufficient because:

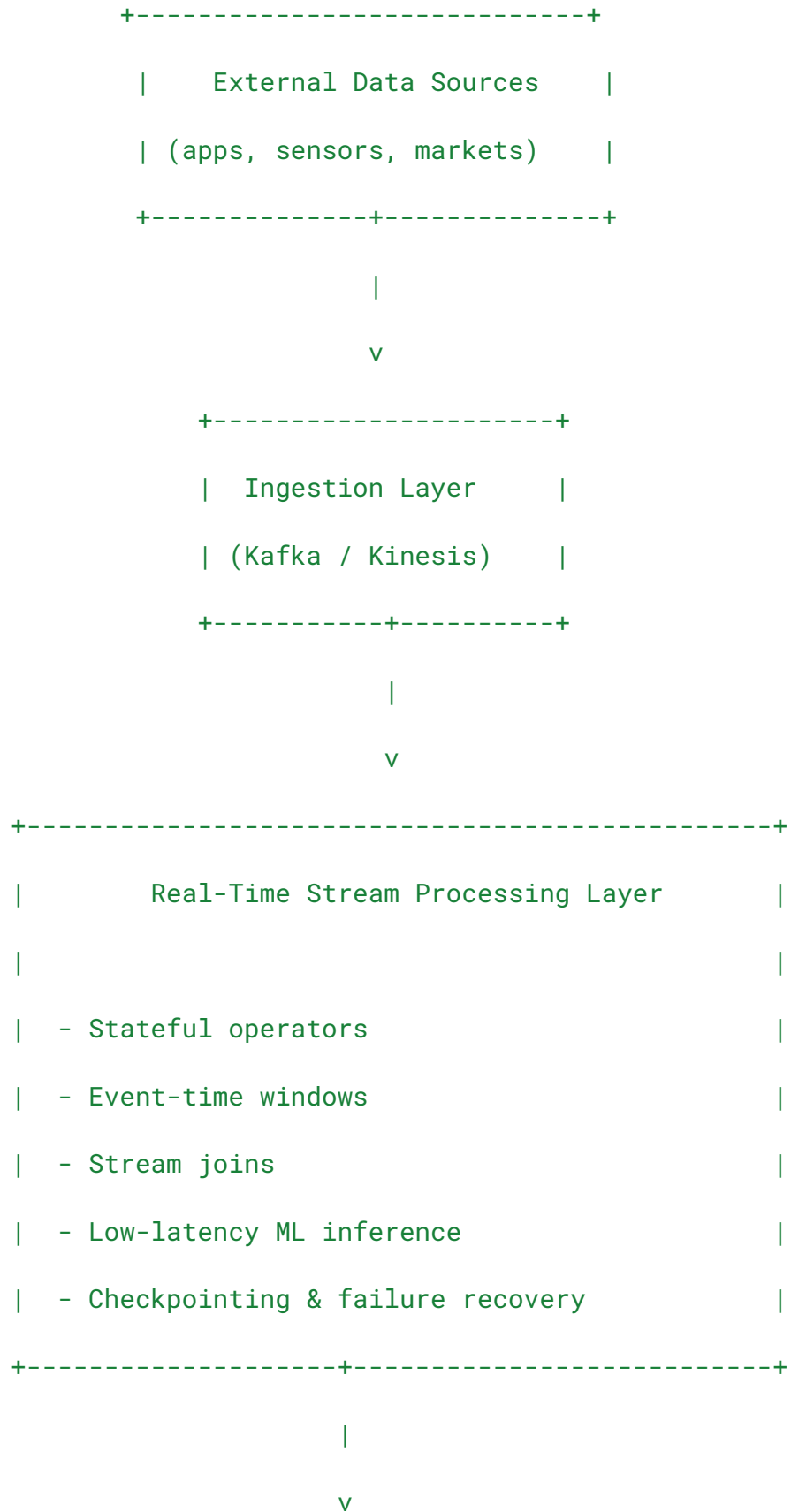
- It processes data hours after arrival.
- Cannot respond to real-time events (fraud, alerts, anomalies).
- Lacks real-time windows and stateful streaming ability.

RTAP closes this gap by supporting:

- **Continuous transformations:** map, filter, joins, aggregations.
 - **Windowing:** tumbling, sliding, session, decaying.
 - **Incremental ML scoring:** real-time model inference.
-

3. RTAP Architecture (Deep Internal Structure)

A conceptual architecture:





4. Key RTAP Components (Depth)

A) Ingestion Layer

Guarantees:

- durability (log-based architecture)
- partitioning for parallelism
- ordering guarantees per partition

B) Compute Layer

Implements:

- stream operators (map/filter)
- keyed state (per key memory)
- window aggregations (tumbling/sliding/session)
- watermarking (handling late events)
- exactly-once semantics

C) State Storage

RTAP systems run stateful operators with local or remote state:

- RocksDB state backends (Flink)
- Redis for fast counters
- Cassandra/HBase for large distributed state

D) Output/Serving Layer

- Real-time dashboards
- Alerts (fraud detection, anomaly detection)
- APIs for applications

5. RTAP Applications (Deep Coverage)

1) Fraud Detection

- Continuous scoring of transactions

- Features computed over sliding & decaying windows
- Online ML models detect abnormal patterns in real time

2) IoT Sensor Monitoring

- Temperature, pressure, humidity streams
- Real-time anomaly detection
- Time-decayed windows for smooth metrics

3) User Behavior Analytics

- Clickstreams
- Real-time recommendation
- Real-time funnel analytics

4) Operational Monitoring

- Logs, metrics, latency
- Alerting based on stream metrics

5) Ad-tech (Real-Time Bidding)

- Millisecond latency decisions
- High throughput

6) Financial Market Analytics

- Price movement analysis
- Volatility estimation

- Trend detection using windowed aggregations

2) CASE STUDY: REAL-TIME SENTIMENT ANALYSIS

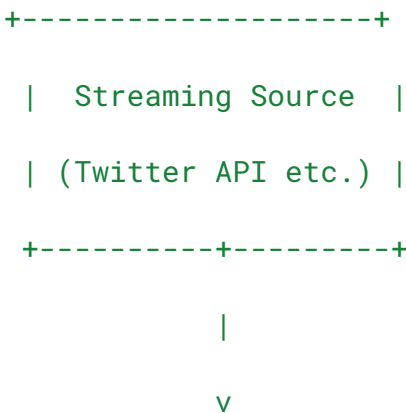
1. Definition

Real-time sentiment analysis identifies the emotional polarity (positive/negative/neutral) of text **as it streams in** from sources such as:

- Twitter (X)
- product reviews
- social media live comments
- news feeds

Goal:
Measure public reaction *instantly*.

2. Pipeline Architecture (Deep Explanation)





3. Techniques Used in Real-Time Sentiment

A) Lexicon-Based

- Dictionary of words with sentiment scores
- Fast, deterministic, minimal CPU

Process:

1. Tokenize text
2. For each word, lookup sentiment score
3. Aggregate → compute polarity

B) Machine Learning-Based

- Logistic Regression
- SVM
- Neural models

Constraints:

- Must be lightweight to score in real time
- Often deployed as ONNX or TensorFlow Lite models

C) BERT-like Models (optimized)

- DistilBERT, MiniLM
- Must run on GPU or optimized CPU inference

4. Windowed Sentiment Metrics

Real-time pipelines compute metrics such as:

`sentiment_avg_10s` = average sentiment over last 10 seconds

`top_positive_topics_1min` = most positive keywords over last 1 min

`sentiment_trend` = slope of sentiment over sliding window

Event-time windows ensure correctness even for late-arriving tweets.

5. Applications

- Brand monitoring
 - Political event analysis
 - Real-time response to crises
 - Market sentiment signals
 - Trend prediction
-

3) CASE STUDY: STOCK MARKET PREDICTIONS

1. Definition

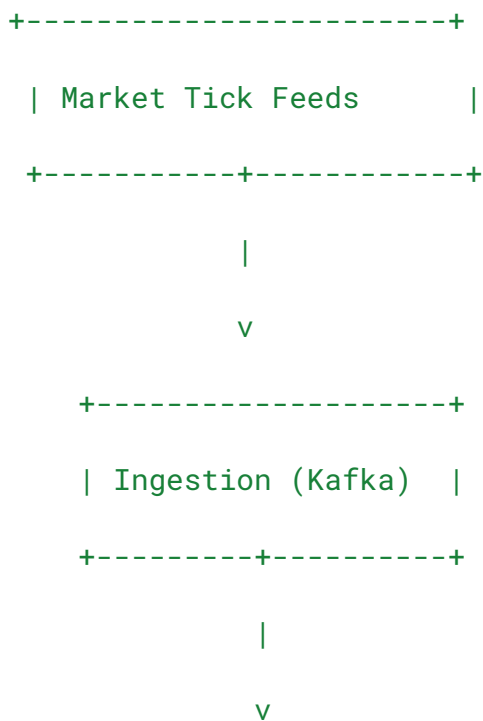
Real-time stock prediction involves:

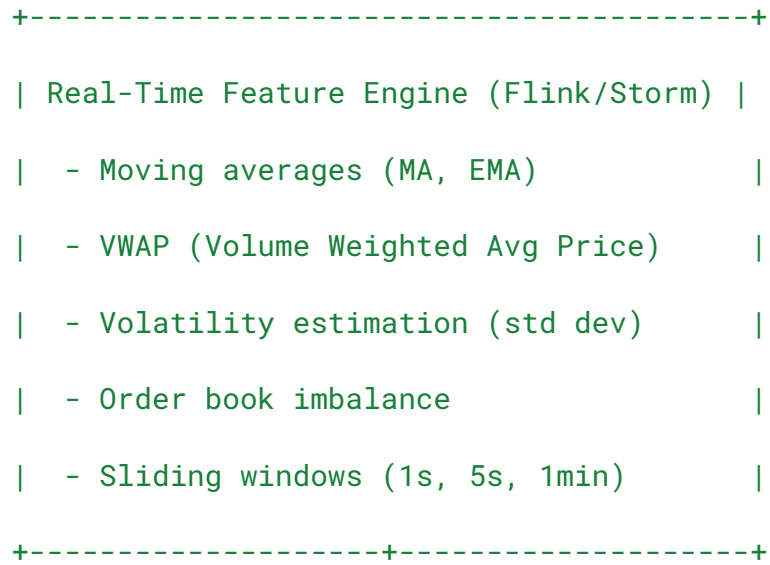
- ingesting live market tick data
 - computing features on the fly
 - predicting price movement or volatility
 - generating alerts/orders with minimal latency
-

2. Market Data Characteristics

- very high frequency (microsecond updates)
 - low latency requirements (< 10 ms)
 - multiple sources (tick data, order book snapshots, news)
-

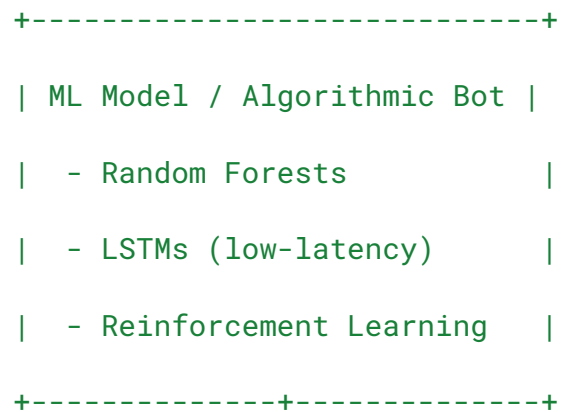
3. Real-Time Stock Analytics Pipeline





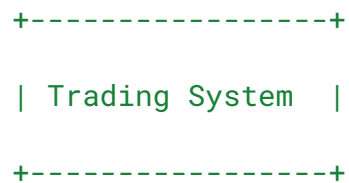
|

v



|

v



4. Real-Time Features (Deep Definitions)

A) Moving Average (MA)

$$MA(t) = (p[t] + p[t-1] + \dots + p[t-n+1]) / n$$

B) Exponential Moving Average (EMA)

$$EMA(t) = \alpha * p[t] + (1 - \alpha) * EMA(t-1)$$

(where α is smoothing factor)

C) Volatility (windowed std deviation)

$$\sigma = \sqrt{(\sum(p - \mu)^2) / N}$$

D) Order Book Imbalance

$$OBI = (BidVolume - AskVolume) / (BidVolume + AskVolume)$$

5. ML Approaches

A) Tree Models (RF, XGBoost)

- Fast scoring
- Good for tabular tick features

B) Recurrent Neural Networks (LSTM/GRU)

- Used in high-frequency modeling

- Often require heavy optimization

C) Reinforcement Learning

- Agents that learn trading strategies dynamically
-

6. Use Cases

- Short-term price prediction
 - Volatility forecasting
 - Arbitrage detection
 - High-frequency trading signals
 - Risk monitoring
-

4) USING GRAPH ANALYTICS FOR BIG DATA

1. Definition

Graph analytics is the study of relationships between entities using graph structures:

- vertices (nodes) represent entities
- edges represent connections or interactions

Graph analytics on big data means processing massive graphs (millions/billions of nodes and edges) using distributed or streaming systems.

2. Why Graph Analytics is Needed

Streaming/big data contexts often have:

- social interactions (Twitter users, followers)
- transaction networks (fraud rings)
- IoT network relationships
- web links
- product-user connectivity (recommendation systems)

Graph structures reveal:

- communities
 - influence
 - anomalies
 - shortest paths
 - centrality
-

3. Graph Processing Models

A) Batch Graph Processing

Systems: Apache Giraph, GraphX (Spark), Pregel model.

Characteristics:

- good for static graphs
- iterative vertex computation

B) Streaming Graph Analytics

Graph evolves over time:

- new edges/nodes arrive
- graph properties must update dynamically

Examples:

- detecting new fraud rings as transactions arrive
- updating PageRank incrementally
- evolving community detection

C) Distributed Graph Databases

Neo4j, JanusGraph, TigerGraph.

4. Important Graph Algorithms (Deep Definitions)

A) PageRank

Measures node importance based on link structure:

$$PR(v) = (1 - d)/N + d * \sum (PR(u) / \text{outdegree}(u))$$

with damping factor d (≈ 0.85).

B) Connected Components

Find groups of nodes reachable from one another.

C) Community Detection (Louvain)

Find clusters of densely connected nodes.

D) Centrality Measures

- Degree centrality = number of edges
- Betweenness centrality = number of shortest paths through node
- Closeness centrality = average distance to all nodes

E) Shortest Path (BFS/Dijkstra)

Used for routing, dependency analysis.

5. Graph Analytics Applications (High Depth)

1) Fraud Detection

- Fraud rings are graph communities
- Circular transaction patterns
- Use PageRank-like scoring to detect suspicious nodes

2) Social Network Analysis

- Influence ranking
- Community finding
- Virality prediction

3) Recommendation Systems

- Graph embeddings
- Collaborative filtering as bipartite graph

4) Cybersecurity

- Network intrusion modeled as anomalous subgraphs

5) Knowledge Graphs

- Semantic search
 - AI reasoning
-

6. Challenges in Big Graph Analytics

- Large memory footprint
- Heavy communication in distributed algorithms
- Dynamic updates more complex than static batch processing
- Maintaining correct state under streaming edge arrivals

PRACTICE QUE ON UNIT-5

1. QUESTION 1

Explain the concept of a data stream. How does the stream processing model differ from the traditional batch processing model?

Detailed Answer

A **data stream** is an unbounded, continuous, time-ordered sequence of data items arriving at high velocity from sources such as sensors, clickstreams, social media, logs, and financial markets. Stream systems process data **on the fly**, usually in **one pass**, with **sub-linear memory**.

Key characteristics:

- Unbounded length
- Real-time arrival

- Memory constraints
- Often approximate computation
- Order may be out-of-order (event-time vs processing-time)

Difference from Batch Processing

Aspect	Batch Processing	Stream Processing
Data Size	Finite datasets	Infinite/unbounded
Processing	After data collection	Real time, continuous
Latency	High (minutes/hours)	Low (ms–seconds)
Memory	Can scan multiple times	Must use sketches
Accuracy	Exact	Often approximate
Use cases	ETL, reporting	Fraud detection, alerts

Batch processing reads stored data many times. Stream processing uses **one-pass online algorithms**, requiring approximation methods like sketches, probabilistic estimates, and windowing.

2) QUESTION 2

Describe the Stream Data Model and Architecture. Explain ingestion, processing, windowing, and storage layers.

Detailed Answer

A **stream data architecture** models unbounded event flow and defines how data moves and is processed in real time.

Key Components of the Model

1. Stream as a sequence

Stream $S = \{ (value, timestamp) \}$

Items may be late; processing must account for event time.

2. Event-time vs Processing-time

- *Event-time*: time attached to the event by source
- *Processing-time*: time event is seen by system

Windowing often uses event-time for correctness.

Architecture Layers

1. Ingestion Layer

- Systems: Kafka, Kinesis, Pulsar
- Provides durability, partitioning, high throughput
- Writes logs of events, enabling replay and ordering guarantees

2. Processing Layer

- Stateful stream processing: Flink, Spark Streaming, Storm
- Tasks: filtering, mapping, joins, aggregations
- Time-based windowing: tumbling, sliding, session
- Watermarking for late events
- Exactly-once semantics via checkpointing

3. State Storage Layer

- Stores operator state, keyed state

- Systems: RocksDB, Redis, Cassandra

4. Serving Layer

- Dashboards, APIs, alerts
- Real-time metrics, ML scoring output

This architecture enables high-throughput, low-latency real-time analytics.

3) QUESTION 3

What is Stream Computing? Explain its need, characteristics, and example applications in detail.

Detailed Answer

Stream computing refers to continuous processing of real-time data streams using low-latency algorithms that work under strict memory and time constraints.

Characteristics

- **Continuous, never-ending computation**
- **Low latency (ms–seconds)**
- **Distributed parallel execution**
- **Fault tolerance through checkpoint & replay**
- **Window-based computation**
- **Approximate algorithms** (sketches, probabilistic models)

Why Needed?

Large-scale systems (finance, IoT, social media) produce data faster than batch models can process. Immediate insights are critical for:

- **Fraud detection**

- **Real-time bidding**
- **Healthcare monitoring**
- **Stock market analysis**
- **Smart grids and IoT**

Stream computing enables instant reactions rather than waiting for batch cycles.

4) QUESTION 4

Explain Sampling in Data Streams. Discuss reservoir sampling (Algorithm R) with a numerical example.

Detailed Answer

Sampling in streams is extracting a representative subset of elements from an unbounded stream without storing all data.

Reservoir Sampling (Algorithm R)

Goal: Sample **k** items uniformly from a stream of unknown length **n**.

Steps:

1. Fill reservoir with first **k** elements.
2. For each incoming item at index **i** ($i > k$):
 - Generate random $r \in [1, i]$
 - If $r \leq k$, replace $\text{reservoir}[r]$ with item **i**

Numerical Example (k = 2)

Stream: A, B, C, D, E

- Reservoir = [A, B]

- $i = 3 \rightarrow$ choose $r \in [1,3]$; assume $r=3 \rightarrow$ ignore
- $i = 4 \rightarrow r \in [1,4]$; assume $r=1 \rightarrow \text{reservoir}[1]=D \rightarrow [D, B]$
- $i = 5 \rightarrow r \in [1,5]$; assume $r=2 \rightarrow \text{reservoir}[2]=E \rightarrow [D, E]$

Final sample = $[D, E]$ (uniform among all pairs)

Memory: $O(k)$

Time per item: $O(1)$

Sampling allows approximate analytics without storing all data.

5) QUESTION 5

Explain Filtering in Streams. How do Bloom Filters support stream filtering? Provide formulae for false positive probability.

Detailed Answer

Filtering selects items of interest from a stream using predicates (e.g., keep only error logs, packets from certain IPs).

Traditional filtering needs entire sets stored in memory.

Bloom Filter allows memory-efficient membership testing with controlled false positives.

Structure

- Bit array of size m
- k independent hash functions

Operations

Insert(x):

For all $i \in [1, k]$, set $\text{bit}[h_i(x)] = 1$

Query(x):

If all bits at positions $h_i(x) = 1 \rightarrow$ maybe present

Else $\rightarrow$ definitely absent

False Positive Probability

$$p = (1 - (1 - 1/m)^{(k*n)})^k$$

Approximation:

$$p \approx (1 - e^{(-kn/m)})^k$$

Choosing optimal k:

$$k_{opt} = (m/n) * \ln(2)$$

Advantages

- Constant-time $O(k)$ operations
- Very low memory usage
- Supports high-throughput streams

Disadvantage

- False positives
- No deletion (unless using Counting Bloom Filter)

6) QUESTION 6

Explain distinct element estimation in data streams. Describe HyperLogLog algorithm, data structures, and error guarantees.

Detailed Answer

Counting distinct values exactly requires $O(N)$ memory. Streaming systems use approximate estimators.

HyperLogLog (HLL)

Industry standard for cardinality estimation.

Core Idea

If hash values are uniformly distributed, the probability of observing r trailing zeros in $\text{hash}(x)$ is $1 / 2^{(r+1)}$.

Rare events (more zeros) indicate large cardinality.

Algorithm

1. Hash every element to a 32-bit or 64-bit number.
2. Divide hash space into m registers using first b bits; $m = 2^b$.
3. For each register j , store $R[j] =$ maximum number of trailing zeros seen.
4. Estimate:

$$E = \alpha_m * m^2 / \sum (2^{(-R[j])})$$

α_m is a bias correction constant.

Error Guarantee

Relative error $\approx 1.04 / \sqrt{m}$

With $m = 2048$ registers, relative error $\approx 2.3\%$.

Memory

A few KB — extremely efficient.

Used in:

Google Analytics, Redis, Amazon Redshift, Cassandra.

7) QUESTION 7

What are frequency moments? Explain F_0 , F_1 , F_2 with the AMS Sketch for F_2 .

Detailed Answer

For stream S , let $f_i =$ frequency of element i .

Frequency Moments

- F_0 = number of distinct elements
- F_1 = total length of stream
- $F_2 = \sum f_i^2$ (used for detecting skew, heavy hitters)

AMS Sketch for F_2

Uses random ± 1 hash function $g(i)$. Maintain:

$Z = \sum g(x_t)$ for all stream items

Estimator:

$$F2_est = Z^2$$

To reduce variance, maintain t independent copies:

$$F2_est = \text{average}(Z_j^2)$$

Why it works

$E(Z^2) = \sum f_i^2 = F_2$ (due to cancellation of cross terms because $g(i)$ is random)

Memory: $O(1/\epsilon^2 \log(1/\delta))$

Update time: $O(1)$

This sketch is foundational in streaming theory.

8) QUESTION 8

Explain Sliding Window Counting. Describe DGIM Algorithm with an example and memory analysis.

Detailed Answer

Problem: Count number of 1s in last N bits.

Naive solution: store N bits $\rightarrow O(N)$ memory (too large).

xDGIM Algorithm

Stores buckets of size: 1, 2, 4, 8, ... powers of 2.

Rules:

- Each bucket has size (power of 2) and timestamp of its newest bit.
- At most **two buckets per size** allowed.
- When a third occurs → merge two oldest into one bigger bucket.
- Buckets older than window N are dropped.

Estimation

Count = sum of bucket sizes

Minus half of the oldest bucket (since boundaries unknown)

Accuracy

≤ 50% relative error (can be improved with modifications)

Memory

$O(\log^2 N)$

Example (brief)

Stream window of last 16 bits:

1 0 1 1 0 0 1 1 1 0 1 0 0 1 1 1

Buckets might be:

[8], [4], [2], [1], [1]

Sum sizes = 16

Subtract half oldest ($8/2=4$) → approx count = 12

Actual = 11 (illustrates approximation)

9) QUESTION 9

Explain Decaying Window Model. Compare exponential decay with fixed sliding windows.
Provide formulae.

Detailed Answer

A **decaying window** gives more weight to recent data without sharply discarding older data.

Weight Function

For event at time t_i , current time T :

$$w_i = \exp(-\lambda * (T - t_i))$$

λ = decay rate

Properties

- λ high $\rightarrow$ fast decay
- Smooth weighting vs abrupt window drop
- Memory efficient
- Suitable for trending metrics

Aggregates

Weighted sum:

$$S(T) = \sum w_i * x_i$$

Weighted average:

$$\text{Avg}(T) = S(T) / \sum w_i$$

Comparison with Sliding Window

Feature	Sliding Window	Decaying Window
Boundary	Hard cutoff	Smooth decay
Memory	Needs window state	Lightweight
Sensitivity	Sudden changes	Gradual fading
Use cases	recent-only metrics	trend analysis

Used in:

anomaly detection, adaptive algorithms, stock price smoothing.

10) QUESTION 10

Explain Real-Time Analytics Platform (RTAP) with case studies: Real-Time Sentiment Analysis and Stock Market Prediction.

Detailed Answer

RTAP enables immediate insights by processing live event streams.

Components

- Ingestion: Kafka
 - Real-time processing: Flink, Spark Streaming
 - Windowing and event-time handling
 - Online ML scoring
 - State store: RocksDB, Redis
 - Output: dashboards, alerts
-

Case Study A: Real-Time Sentiment Analysis

Pipeline:

1. Ingest tweets in Kafka
2. Stream processor cleans text
3. Apply sentiment model (lexicon or ML)
4. Compute metrics in sliding windows (10s, 1min)
5. Output live sentiment dashboard

Applications:

- brand monitoring
 - election analysis
 - crisis management
-

Case Study B: Stock Market Prediction

Pipeline:

1. Ingest market ticks
2. Compute features in sliding/decaying windows:
 - EMA
 - volatility
 - order book imbalance
3. ML model (RF/LSTM) predicts price movement
4. Automated trading engine triggers orders

Applications:

- high-frequency trading
 - volatility forecasting
 - arbitrage detection
-

Graph Analytics in RTAP

Used for:

- fraud rings
- social network influence
- recommendation systems

Algorithms:

- PageRank
- Connected components
- Community detection

Graph analytics complements streaming by uncovering **relationships**, not just events.

UNIT -6 NOSQL DB

1.SCHEMA-LESS MODELS

Definition:

A schema-less (or schema-flexible) database allows records to have different structures without requiring a predefined fixed schema. Each record may have its own set of fields, nesting, data types, or hierarchy.

Why Schema-less Models?

- Traditional RDBMS require strict schema definitions (tables, columns, constraints).
- Big Data systems deal with rapidly evolving data (IoT, logs, social media).
- Schema evolution (adding new fields, changing types) is extremely costly in relational systems.

Key Characteristics:

1. **Flexibility** — structure changes without expensive ALTER TABLE operations.
2. **Polyglot data** — store structured, semi-structured, or unstructured data.
3. **Agility for developers** — modify application-level structure without DBA intervention.
4. **High write throughput** — ideal for log streams, sensor data, event stores.
5. **Horizontal scalability** — easily partition records across distributed nodes.

Use Cases:

- JSON logs
- User profiles (each user may have different attributes)
- Product catalogs (different products have different fields)
- Configuration stores
- Content management systems

Example (JSON-like document with no required fields):

```
{  
  "user_id": 10294,  
  "name": "Arjun",  
  "preferences": { "theme": "dark" }  
}
```

Another record may be:

```
{  
  "user_id": 88901,  
  "email": "xyz@gmail.com",  
  "purchases": ["Laptop", "Mouse"]  
}
```

No rigid schema — the database does not enforce field names or data types.

2. KEY–VALUE STORES

Definition:

A key–value store is the simplest NoSQL model where each item is stored as:
(Key $\rightarrow$ Value)

The database retrieves values directly by their key.

Characteristics:

- $O(1)$ average-time lookup (hash-based indexing)
- Value can be any opaque blob (JSON, XML, binary, images, videos)
- No joins, no relations
- Extremely high write/read throughput
- Scales horizontally across distributed clusters

Internal Structure:

- Typically implemented using distributed hash tables (DHT)
- Data partitioned by hashing key
- Simple read/write API (get, put, delete)

Strengths:

- Ultra-fast access for session data, caches
- Perfect for large-scale caching systems
- Easy scalability

Limitations:

- No querying on values
- No secondary indexes (unless extended implementation)
- No complex transactions or relationships

Examples:

- Redis
- Amazon DynamoDB
- Riak
- Memcached

Use Cases:

- Caches (session storage, tokens)
- Shopping carts
- Leaderboards
- Real-time counters
- Configuration data

3. DOCUMENT STORES

Definition:

Document databases store data as **semi-structured documents** (usually JSON or BSON). Each document has a flexible structure and may contain nested objects and arrays.

Structure:

- Document → collection of key-value pairs
- Can be nested or hierarchical
- Collections ≈ tables (but schema-free)

Example Document:

```
{  
  "product_id": 301,  
  "name": "Laptop",
```

```
"specs": { "ram": "16GB", "cpu": "i7" },  
"tags": ["electronics", "computers"]  
}
```

Advantages:

1. **Schema flexibility**
2. **Rich querying** — queries on nested fields
3. **Indexing** — secondary indexes supported
4. **Aggregation pipelines**
5. **Good for hierarchical data**

Optimized for:

- CRUD operations
- JSON-based APIs
- Large-scale content storage
- Microservices backend systems

Limitations:

- Joins are limited or absent (though some support “lookup” operations)
- Large documents lead to overhead

Examples:

- MongoDB
- CouchDB
- RavenDB
- Elasticsearch (document indexing)

Use Cases:

- User profiles
- Product catalogs
- Blog posts/articles
- Content management systems
- Event logs

4. TABULAR STORES / WIDE-COLUMN STORES

Definition:

Wide-column stores (also called tabular or column-family stores) organize data into **column families** instead of fixed columns like RDBMS.

Important distinction:

- RDBMS → fixed schema, columns defined upfront
- Wide-column → flexible, sparse columns per row; each row can have different columns

Internal Structure:

A Column Family contains:

Row Key → (Column Name : Value) pairs

Columns can be grouped into “families”.

Example (Cassandra/HBase-like):

Row key: u123

Columns:

name → "Anita"

age → 29

last_login → "2025-06-21"

Another row may have totally different columns.

Characteristics:

- Optimized for massive write throughput
- Highly distributed, partitioned by row-key
- Tunable consistency (eventual vs strong)
- Extremely scalable (petabyte-scale)

Strengths:

- Efficient range queries
- Good for time-series data
- Sharding built-in
- Supports high ingestion rates (IoT, clickstreams)

Limitations:

- Complex data modeling
- No ad-hoc query flexibility like SQL
- No joins or complex transactions

Examples:

- Apache Cassandra
- Apache HBase
- Google Bigtable (origin of the model)
- ScyllaDB

Use Cases:

- Messaging services
- Time-series databases

- IoT telemetry systems
- Write-heavy analytics platforms
- Distributed logging systems

5.OBJECT DATA STORES

Definition:

Object databases store data as **programmable objects**, not rows or documents.

Objects include both data and behavior (methods).

They follow object-oriented programming (OOP) principles.

Object = attributes + methods

Stored directly without mapping to relational tables.

Motivation:

In traditional systems, developers convert objects → tables using ORM (Object-Relational Mapping).

Object databases eliminate this mismatch.

Characteristics:

- Designed for OOP languages (Java, C++, Python)
- Supports inheritance, encapsulation, polymorphism
- Objects stored with references/links
- No SQL — access is via object manipulation APIs

Strengths:

1. Direct mapping between application objects and DB objects
2. No impedance mismatch
3. Good for graph-like or hierarchical structures

4. Ideal for simulation, engineering, CAD systems

Limitations:

- Querying is less flexible than document or relational databases
- Harder to scale horizontally compared to key-value/document stores
- Limited tooling and ecosystem
- Not widely used in large-scale web systems

Examples:

- db4o
- ObjectDB
- Versant Object Database

Use Cases:

- Scientific computing
- CAD/CAM
- Embedded systems
- Object-heavy enterprise applications

6. GRAPH DATABASES

Definition:

A **graph database** is a NoSQL database designed to store and query data represented as **nodes**, **edges**, and **properties**.

It uses graph theory to model relationships as first-class citizens.

Data Model:

- Node: entity/object (e.g., Person, Product, City)
- Edge: directed or undirected relationship between nodes (e.g., "FRIEND", "LIKES", "LOCATED_IN")
- Property: key-value information stored on nodes or edges
- Graph: a set of interconnected nodes and edges

Why Graph Databases?

Relational databases struggle with multi-hop joins (e.g., friend-of-friend-of-friend).

Graphs support traversal in $O(k)$ time (k = traversal depth), not $O(n^3)$ like relational joins.

Key Characteristics:

- Schema-optional
- Index-free adjacency (each node holds direct pointers to neighbors)
- High performance for deep relationship queries
- Supports graph algorithms: PageRank, community detection, shortest path, centrality

Graph Query Languages:

- Cypher (Neo4j)
- Gremlin (Apache TinkerPop)
- SPARQL (RDF graphs)

Example Graph Structure (Friendship):

Node1: {id:1, name:"A"}

Node2: {id:2, name:"B"}

Edge: (1)-[:FRIEND]->(2)

Use Cases:

- Social networks

- Recommendation engines
- Fraud detection
- Knowledge graphs
- Network/IT dependency graphs

Popular Systems:

- Neo4j
- Amazon Neptune
- JanusGraph
- OrientDB
- TigerGraph

7.HIVE

Definition:

Apache Hive is a **data warehousing platform** built on top of Hadoop for querying and managing large datasets using **SQL-like language** called HiveQL.

Key Idea:

Hive converts SQL queries → MapReduce jobs (or Tez/Spark in newer versions).

Characteristics:

- Schema-on-read (flexible)
- Handles structured/semi-structured data
- Designed for batch processing
- High latency (seconds/minutes)

Architecture Components:

- HiveQL Engine: parses queries
- Driver + Compiler: converts queries to execution plan
- Metastore: stores table schemas, partitions
- Execution Engine: executes jobs using Hadoop or Spark
- Storage: HDFS, S3, Azure Blob, etc.

Table Types:

- Managed tables
- External tables (data stored outside Hive's control)
- Partitioned tables (faster queries)

Strengths:

- SQL-like interface for Hadoop
- Handles petabyte-scale data
- Integrates with BI tools

Limitations:

- Not suitable for low-latency queries
- Not a NoSQL database; it is a "SQL-on-Hadoop" engine

Examples of Use Cases:

- Log analysis
- Data warehousing
- ETL pipelines in big organizations

- Large-scale analytical dashboards

8. SHARDING (HORIZONTAL PARTITIONING)

Definition:

Sharding is the process of splitting a large dataset across multiple machines (shards) so that each machine stores only a portion of the data.

Purpose:

- Improve horizontal scalability
- Reduce read/write load on a single node
- Enable parallel processing

How Sharding Works:

Data is divided using:

1. Hash-based sharding (key $\rightarrow$ hash $\rightarrow$ shard)
2. Range-based sharding (value ranges)
3. Directory-based sharding (lookup table maintains shard-to-key mapping)

Shard Key:

A field used to distribute data across nodes (e.g., `user_id`).

Challenges:

- Cross-shard queries require scatter-gather
- Re-sharding (changing shard boundaries) is complex
- Hotspots if shard key is poorly chosen

Sharding in NoSQL Databases:

- MongoDB uses automatic sharding
- Cassandra and HBase use consistent hashing
- DynamoDB uses partition keys automatically

9. HBASE

Definition:

Apache HBase is a **distributed, scalable, column-oriented** NoSQL database built on top of HDFS.

Based on Google Bigtable design.

Data Model:

- Table → column families → columns
- Row key uniquely identifies a row
- Rows are lexicographically ordered
- Supports sparse, variable schema rows

Key Concepts:

- Region: horizontal partition of table
- RegionServer: handles regions (stores data)
- HMaster: manages cluster, region balancing
- Write-Ahead Log (WAL) for fault tolerance
- MemStore + HFile = write optimization

Strengths:

- Real-time read/write
- Horizontal scaling to billions of rows
- Strongly consistent reads/writes
- Tunable performance

Limitations:

- Complex configuration
- No SQL-like interface (unless via Phoenix)
- Poor at ad-hoc querying

Use Cases:

- Time-series data
- Messaging and log ingestion
- IoT data storage
- Facebook's Messaging system
- Search indexing backend

10. BIG DATA CASE STUDIES

A) ANALYZING BIG DATA WITH TWITTER

Data Characteristics:

- High volume (millions of tweets per minute)

- Unstructured (text, emojis)
- Real-time streaming
- Rich metadata (hashtags, mentions, geolocation)

Common Analytics Tasks:

1. Trend detection
2. Sentiment analysis
3. Hashtag popularity tracking
4. User influence ranking
5. Network analysis (retweets, followers graph)

Tools Used:

- Kafka → streaming ingestion
- Spark Streaming / Flink → real-time analytics
- HBase / Cassandra → storing tweet metadata
- Elasticsearch → search over text
- Neo4j → user-graph analysis
- Hive / Presto → batch SQL analytics

Examples:

- Detecting trending topics using sliding windows
- Sentiment classification with ML models
- Spam/fake account detection using graph analytics

B) BIG DATA FOR E-COMMERCE

Data Types:

- User behavior logs
- Product views, clicks, searches
- Transactions, payments
- Product metadata
- Reviews, ratings

Key Analytics:

- Recommendation systems
- Purchase prediction
- Fraud detection
- Dynamic pricing
- Customer segmentation

Big Data Technologies:

- Cassandra / MongoDB for product catalogs
- Redis for cart/session caching
- Spark ML for recommendations
- Kafka for event streams
- ElasticSearch for product search
- Hive/S3 as data lake

C) BIG DATA FOR BLOGS

Data Sources:

- Articles, tags, categories
- Comments, reactions
- User reading history
- Social shares

Use Cases:

- Content ranking
- Trend detection
- Recommendation systems (“similar articles”)
- Spam comment detection
- Personalized homepage

Tools:

- MongoDB → store posts + metadata
- Neo4j → related-content graph
- Elasticsearch → full-text search
- Hadoop/Hive → content analytics

11. REVIEW OF BASIC DATA ANALYTIC METHODS USING R

R is widely used for:

- Statistical modeling

- Data exploration
- Visualization
- Machine learning
- Big data analysis with packages like dplyr, data.table, sparklyr

Core Concepts:

1. Data frames
2. Vectors, lists, matrices
3. Functions and apply-family
4. Import/export CSV, JSON, databases
5. Plotting (ggplot2)
6. Regression models
7. Classification (SVM, trees, random forest)
8. Clustering (K-means, hierarchical)
9. Time-series analysis (ARIMA, ETS)

Big Data Extensions:

- sparklyr (R + Apache Spark)
- rhadoop
- bigmemory & ff (handle large datasets)

Example R Workflow:

- Import data
- Clean missing values

- Transform (dplyr)
- Build model (lm, glm, randomForest)
- Evaluate (confusion matrix, RMSE)
- Visualize

COMPARISON OF DB

Type	Data Model	Schema	Strengths	Weaknesses	Examples	Use Cases
Key-Value Store	Key → Value	Schema-less	Fast lookups, caching, simple API	No querying, no relations	Redis, DynamoDB, Riak	Sessions, tokens, caching
Document Store	JSON/BSON documents	Flexible schema	Rich queries, nested structures	Not ideal for complex joins	MongoDB, CouchDB, Elasticsearch	User profiles, catalogs, CMS
Wide-Column Store	Column families, sparse rows	Semi-flexible	Massive scale, high write throughput	Hard modeling, limited querying	Cassandra, HBase, Bigtable	Logs, time series, IoT
Graph Database	Nodes + Edges + Properties	Schema-optional	Relationship queries, graph traversal	Hard to scale for writes	Neo4j, JanusGraph, Neptune	Social networks, fraud detection
Object Database	Objects with attributes + methods	Depends on object	Natural for OOP apps	Weak querying, low adoption	db4o, ObjectDB	CAD, scientific apps

UNIT 6 PRACTICE QUESTIONS

1. What is Hive? Explain its architecture, components, and how it enables SQL-like querying on Hadoop.

Answer:

Apache Hive is a data warehousing and SQL-like query system built on top of Hadoop. It allows users to perform **ad-hoc queries**, **summaries**, and **analysis** of large datasets stored in HDFS using a language called **HiveQL**.

Key Concepts

- Not a typical database → It is a **query engine**.
 - Schema is stored in the **Metastore**.
 - Executes queries using **MapReduce / Tez / Spark** engines.
-

Architecture Components

1. Hive Client

- CLI, JDBC, ODBC drivers
- Users enter queries

2. HiveQL Parser

- Parses SQL-like queries into AST (Abstract Syntax Tree)
- Performs semantic analysis

3. Optimizer

- Logical plan optimisation
- Predicate pushdown, projection pruning

4. Execution Engine

- Converts logical plan into physical plan
- Generates MapReduce, Tez, or Spark jobs

5. Metastore

- Stores metadata: table schema, partitions
- Usually backed by MySQL/PostgreSQL

6. HDFS Storage

- Actual data stored in flat files (CSV, ORC, Parquet, JSON)
-

Advantages

- Familiar SQL interface (HiveQL)
- Handles massive datasets
- Integrates with Hadoop ecosystem

Limitations

- High latency (batch)
 - Not suitable for real-time analytics
-
-

2. Explain the concept of sharding. Why is it essential in NoSQL databases? Illustrate with examples.

Answer:

Sharding is the process of horizontally partitioning a large dataset into smaller subsets called **shards**, each stored on a different server or node.

Why Sharding?

- Handles **large-scale data** beyond a single server
- Increases **write throughput**
- Reduces **latency** by distributing load
- Supports **horizontal scaling**

Types of Sharding

1. Range-Based Sharding

Data divided by ranges.

Example:

UserID 1-1,000,000 → Shard A

UserID 1,000,001-2,000,000 → Shard B

Pros: easy to implement

Cons: hotspot problem

2. Hash-Based Sharding

$\text{Shard} = \text{hash}(\text{key}) \bmod N$

Pros: evenly distributed

Cons: hard to reshard

3. Geo-Sharding

Data partitioned by geographical region.

Example:

India → Shard 1

US → Shard 2

EU → Shard 3

Sharding in NoSQL

NoSQL DB	Sharding Support	Notes
MongoDB	Auto-sharding	Most popular
Cassandra	Partitioners manage sharding	High scalability
HBase	RegionServers	Table regions can split automatically
Redis Cluster	Key slot hashing	16384 hash slots

3. What is HBase? Describe its architecture, data model, and real-world applications.

Answer:

Apache HBase is a **distributed, column-family, NoSQL database** built on top of **HDFS**.

Key Features

- Random read/write access to big data
 - Horizontally scalable
 - Strong consistency
 - Modeled after Google BigTable
-

Data Model Components

1. **Tables**
2. **Rows** (identified by row keys)
3. **Column families**

4. **Columns (qualifiers)**
5. **Cells (timestamped values)**

HBase stores data in **sorted lexicographical order of row keys**.

Architecture Components

1. **HMaster**
 - Coordinates region servers
 - Load balancing
 2. **RegionServer**
 - Handles read/write requests
 - Stores regions (subset of table)
 3. **ZooKeeper**
 - Manages cluster coordination
 - Provides distributed synchronization
 4. **HDFS**
 - Persistent storage layer
-

Applications

- Facebook's Messaging System
- Time-series databases
- Banking transactional logs

- IoT sensor data
 - Telecom call detail records (CDRs)
-
-

4. Explain how Twitter uses Big Data analytics. Describe data characteristics, pipelines, and analytical tasks.

Answer:

Twitter generates **terabytes of real-time streaming data** such as:

- Tweets
 - Retweets
 - Hashtags
 - Mentions
 - Follower graphs
 - Clickstream logs
-

Data Characteristics

- High velocity
 - Unstructured/free-text
 - Graph-structured (followers network)
 - Highly temporal
-

Twitter Big Data Pipeline

1. Collectors

- Real-time tweet collectors using Finagle

2. Queueing

- Kafka for message streaming

3. Storage

- Hadoop HDFS
- Manhattan (Twitter's distributed DB)
- Redis for caching
- Cassandra for user timelines

4. Processing

- Storm/Flink for real-time
- Hadoop/Spark for batch jobs

5. Analytics

- Trending topics
- Spam detection
- User recommendation
- Hashtag clustering

Analytical Tasks

- Sentiment analysis

- Topic modeling (LDA)
 - Graph ranking (PageRank)
 - Real-time trending detection
 - Bot detection using ML
-

5. Explain Big Data applications in E-Commerce with examples.

Answer:

Big Data analytics transforms e-commerce using real-time and predictive insights.

Key Data Sources

- Customer browsing history
 - Clickstream data
 - Purchase logs
 - Product reviews
 - Inventory levels
 - Customer feedback
-

Major Applications

1. Recommendation Systems

- Collaborative filtering (Amazon)

- Content-based filtering
- Hybrid recommenders

Example:

User A bought a phone → Recommend cases, screen guards

2. Dynamic Pricing

ML models adjust price based on:

- Demand
 - Competitor price
 - Seasonality
 - Stock levels
-

3. Fraud Detection

Detects:

- Fake orders
- Account takeover
- Payment fraud

Uses anomaly detection algorithms.

4. Inventory & Supply Chain Optimization

- Predict demand

- Optimize warehouse allocation
 - Reduce shipping time
-

5. Personalized Marketing

- Email campaigns
 - Push notifications
 - Targeted ads
-
-

6. How is Big Data used in blogs? Explain text mining, sentiment analysis, and trend extraction.

Answer:

Blogs generate massive unstructured text.
Big Data tools process this for insights.

Key Techniques

1. Text Mining

- Tokenization
- Part-of-speech tagging
- Named Entity Recognition

Tools: NLTK, SpaCy, Hadoop NLP libraries

2. Sentiment Analysis

Classify blog text into:

- Positive
- Negative
- Neutral

Uses:

- Logistic regression
 - LSTM
 - Transformer models
-

3. Topic Modeling

Clustering using:

- LDA (Latent Dirichlet Allocation)
 - NMF
-

4. Trend Extraction

Using:

- Time-series analysis
 - Keyword frequency tracking
 - Social listening tools
-
-

7. What are basic data analytic methods in R? Explain with syntax and examples.

Answer:

R is widely used for statistical computing and visualization.

1. Descriptive Statistics

```
mean(x)
median(x)
sd(x)
summary(x)
```

2. Data Manipulation (dplyr)

```
library(dplyr)
df %>% filter(age > 20) %>% select(name, salary)
```

3. Data Visualization (ggplot2)

```
ggplot(df, aes(x=age, y=salary)) + geom_point()
```

4. Linear Regression

```
model <- lm(y ~ x1 + x2, data=df)
summary(model)
```

5. Clustering (K-means)

```
kmeans(df, centers = 3)
```

6. Reading Big Data

```
library(data.table)
```

```
fread("bigfile.csv")
```

8. Compare Hive, HBase, and traditional RDBMS with respect to architecture, use cases, schema, and query model.

Answer:

Comparison Table

Feature	Hive	HBase	RDBMS
Type	Data warehouse	NoSQL column store	Relational DB
Query	HiveQL	No SQL, API calls	SQL
Schema	Schema-on-read	Flexible	Schema-on-write
Latency	High (batch)	Low (real-time)	Low/medium
Storage	HDFS	HDFS	Disk
Use Cases	Analytics	Random reads/writes	OLTP
Scalability	Very high	Very high	Limited

9. Explain with a numerical example how sharding improves throughput in a distributed NoSQL system.

Answer:

Assume:

- 1 server can handle **500 writes/second**
- Data requires **3000 writes/second**

Without Sharding

Capacity = 500
Demand = 3000
→ System fails

With 6 Shards

Each shard gets:

$3000 / 6 = 500 \text{ writes/second}$

All shards operate in parallel:

$\text{Total throughput} = 6 \times 500 = 3000 \text{ writes/sec}$

Conclusion

Sharding increases throughput **linearly with the number of shards**, assuming equal load distribution.

10. Explain the architecture of HBase and how data is stored and retrieved using an example.

Answer:

Storage Example

Table: Student
Row key: "S101"
Column family: marks
Columns: maths, science

Stored as:

RowKey: S101
marks:maths → 90 (timestamp)
marks:science → 85 (timestamp)

HBase stores data in **HFiles** on HDFS.

Read Operation

Process:

1. Client → RegionServer
2. RegionServer → MemStore / HFile
3. Returns latest cell based on timestamp

Latency: few ms (real-time)

Write Operation

1. Write Ahead Log (WAL)
 2. MemStore
 3. Flushed to HFile
-

Why Efficient?

- Sorted by row key
- Log-structured merge storage
- Fast random access